Microsoft Defender for DevOps

Introduction

This assignment explores how one can integrate security scanning directly into the development lifecycle by using **Microsoft Defender for DevOps** and the **Microsoft Security DevOps (MSDO)** tooling ecosystem. MSDO orchestrates multiple static analysis and security tools—such as Bandit, BinSkim, ESLint, CredScan, Template Analyzer, Terrascan, and Trivy—to surface code, configuration, and compliance issues early in the software delivery process.

To apply these capabilities in a practical context, the assignment begins with importing a deliberately vulnerable Python/Django application (*VulnDjango*) into both **GitHub** and **Azure DevOps**. One then performs a local **SAST scan with Bandit**, configures the **Microsoft Security DevOps extension in Azure DevOps pipelines**, and sets up the **MSDO GitHub Action** to run automated scans on code changes. Finally, security findings are reviewed through integrated dashboards—including the **DevOps Security Workbook in Defender for Cloud**—to understand how scan results can be centralized for remediation and governance.

Objectives

The main objective of this assignment is to integrate **Microsoft Defender for DevOps** and the **Microsoft Security DevOps (MSDO)** tools into the development lifecycle to identify and remediate security issues early. Specifically, the assignment aims to:

1. Demonstrate how to import and set up a vulnerable application (*VulnDjango*) in **GitHub** and **Azure DevOps** repositories.
2. Perform **static application security testing (SAST)** locally using **Bandit** and integrate MSDO scans within pipelines.
3. Automate code scanning using the **MSDO GitHub Action** and the **Microsoft Security DevOps Azure DevOps extension**.
4. Visualize and analyze discovered vulnerabilities using the **DevOps Security Workbook in Microsoft Defender for Cloud**.
5. Gain hands-on experience with DevSecOps practices by embedding security checks into both local and CI/CD development stages.

Task Breakdown

Task 1: Clone and Setup the VulnDjango Repository

- Import the **VulnDjango** application into your GitHub and Azure DevOps repositories.

- Configure the local development environment (Python/Django) to run the application.

Task 2: Install and Configure Microsoft Security DevOps (MSDO) Locally

- Install the MSDO CLI on your machine.
- Run a **local Bandit SAST scan** on the VulnDjango codebase.
- Review and document vulnerabilities reported by Bandit.

Task 3: Integrate MSDO in Azure DevOps Pipelines

- Install the **Microsoft Security DevOps extension** in your Azure DevOps organization.
- Create or modify an Azure Pipeline (YAML) to include **MSDO scanning tasks**.
- Trigger a pipeline run and collect the security scan results.

Task 4: Configure MSDO GitHub Action

- Add the **Microsoft Security DevOps GitHub Action** to your VulnDjango repository.
- Run the GitHub Action workflow to perform automated security scans on code pushes or pull requests.
- Analyze findings and save logs/screenshots of detected vulnerabilities.

Task 5: Connect GitHub and Azure DevOps to Microsoft Defender for Cloud

- Enable **Microsoft Defender for DevOps** in your Azure subscription.
- Connect both repositories (GitHub and Azure DevOps) to Defender for Cloud for centralized visibility.

Task 6: Review Findings in the DevOps Security Workbook

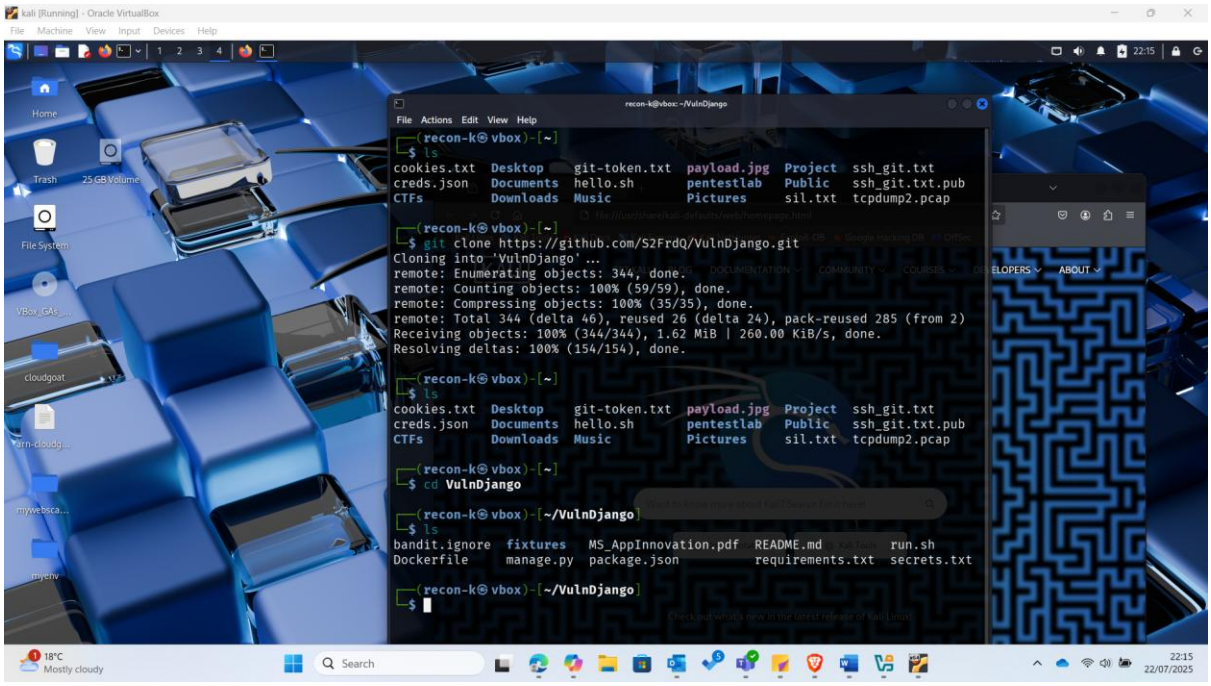
- Open the **DevOps Security Workbook** in Microsoft Defender for Cloud.
- Review reported issues (misconfigurations, secrets, vulnerabilities).
- Document key findings and recommended fixes.

Task 7: Reporting

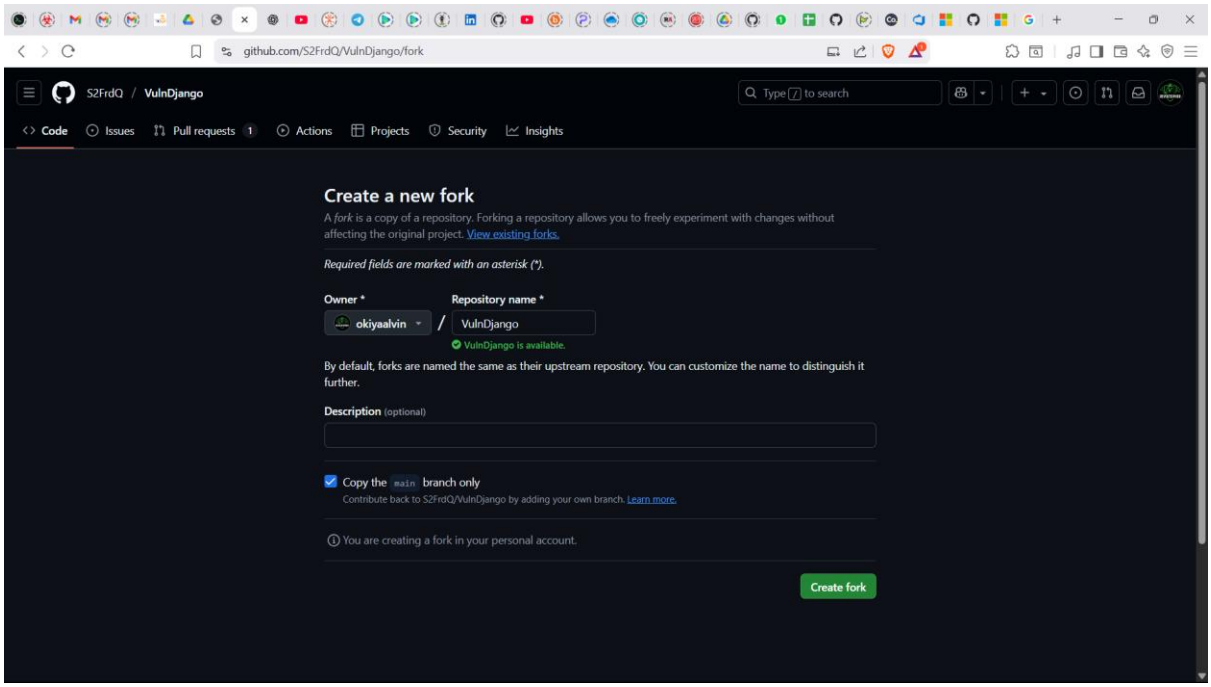
- Compile screenshots, scan results, and workbook findings into the final report.
- Reflect on lessons learned from integrating security into DevOps workflows.

Exercise 1: Import the vulnerable code

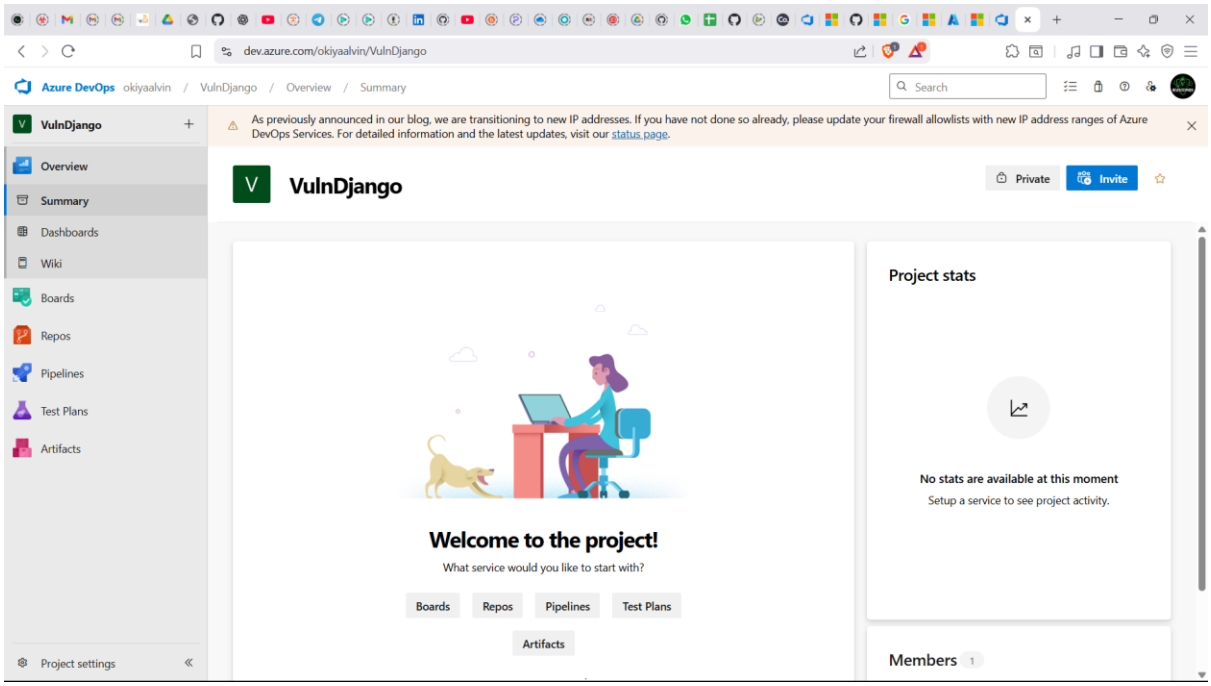
I started by cloning https://github.com/S2FrdQ/VulnDjango.git repository to my kali linux environment.



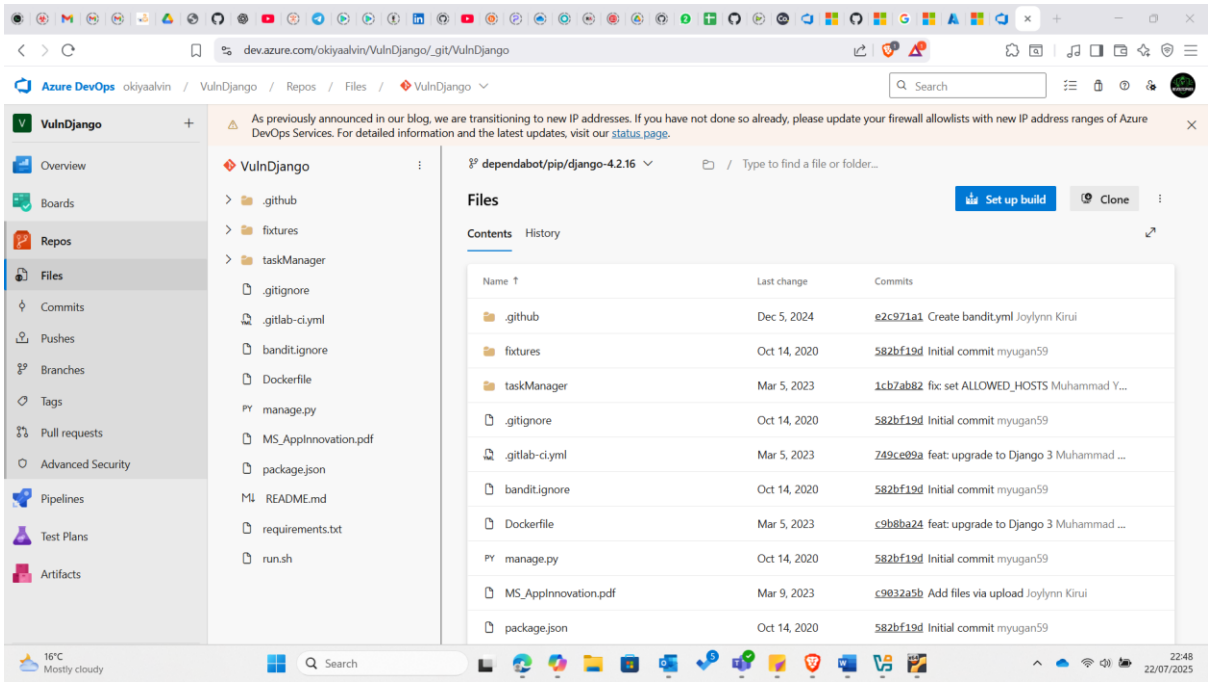
I also forked the repository:



I then created an Azure devops account, using this link https://aex.dev.azure.com/ to avoid redirections to the azure portal. On the azure devops portal, I created a project named VulnDjango:



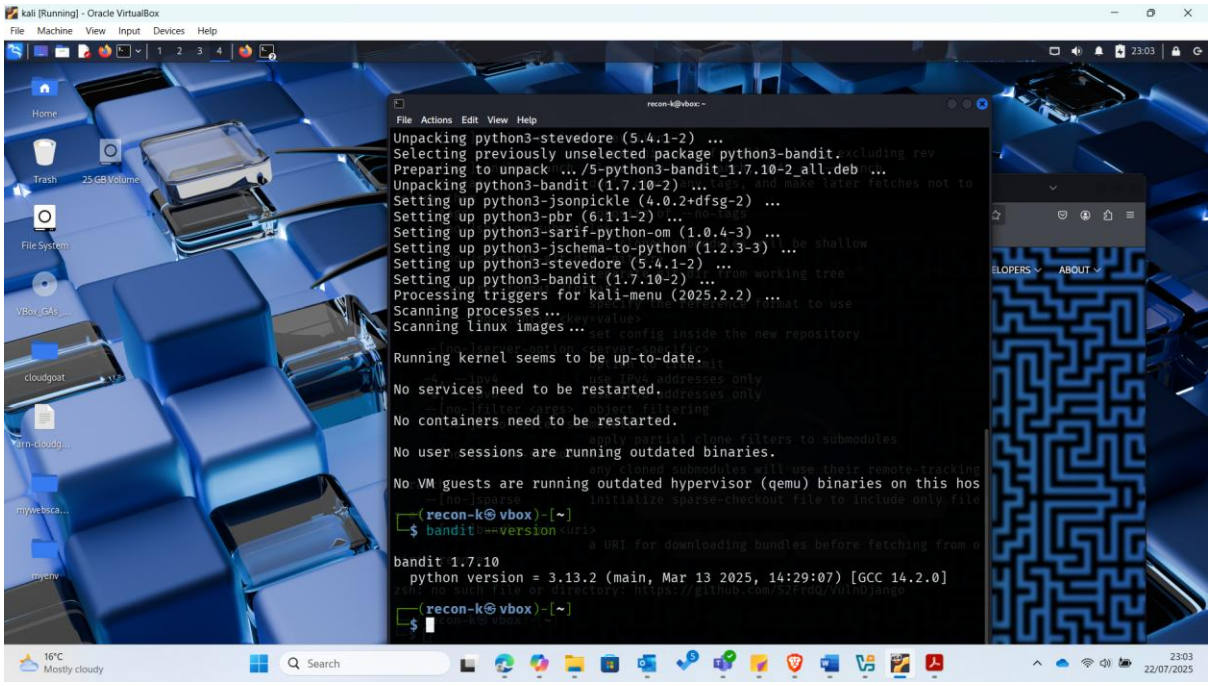
I then navigated to repos at the left blade and imported the vulnerable code from https://github.com/S2FrdQ/VulnDjango.git



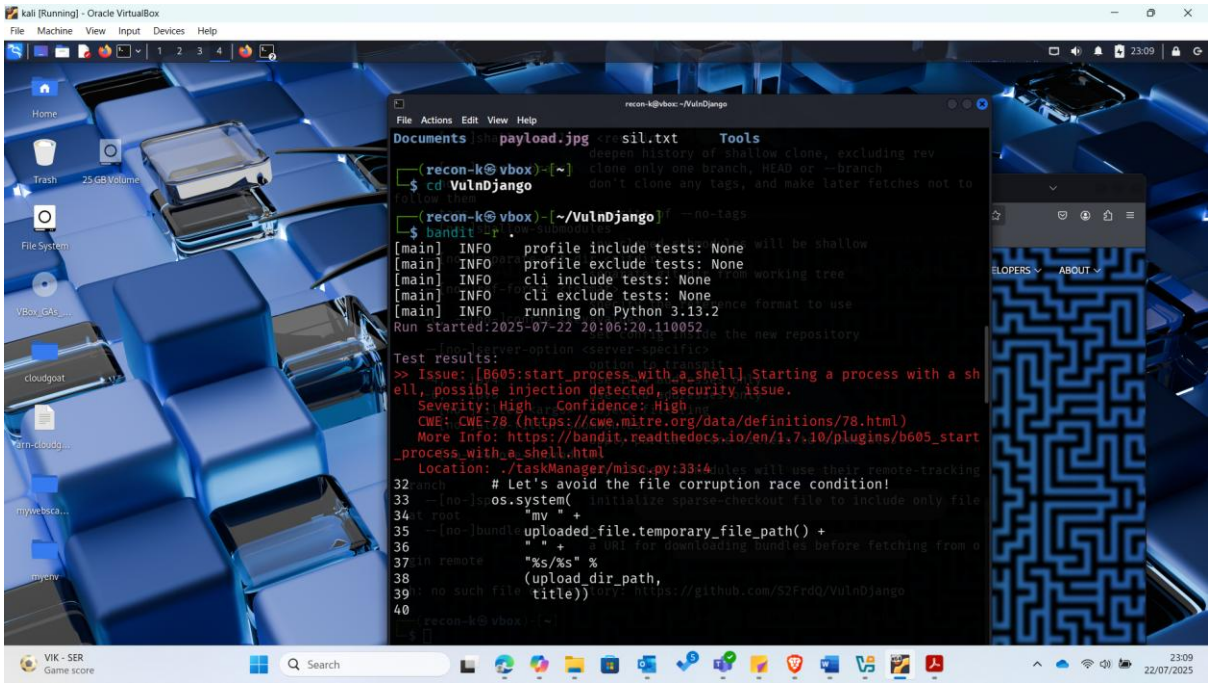
Exercise 2: SAST scan using Bandit locally

Bandit -The Bandit is a tool designed to find common security issues in Python code. To do this Bandit, processes each file, builds an AST, and runs appropriate plugins against the AST nodes. Once Bandit has finished scanning all the files it generates a report. Bandit was originally developed within the OpenStack Security Project and later rehomed to PyCQA.

On my Kali linux machine, since I cloned earlier the VulnDjango repository on my first step, I proceeded to install bandit using the command: sudo apt install python3-bandit

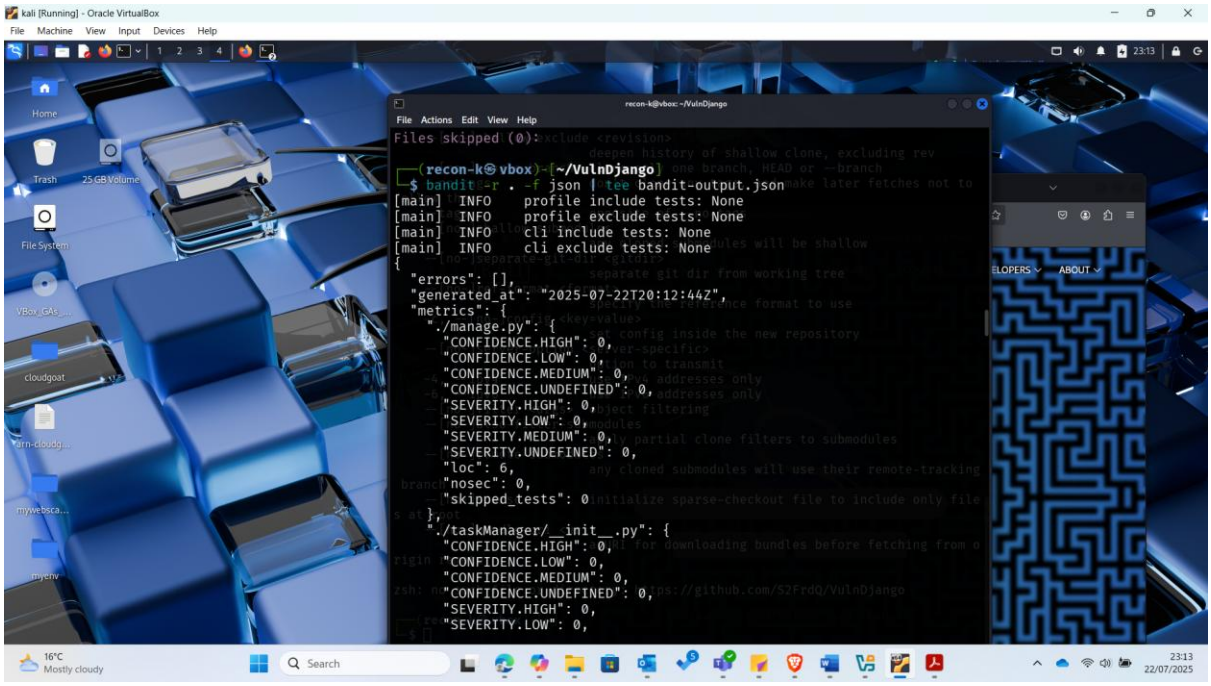


After successful installation and checking the bandit version so as to confirm whether the installation was successful, I navigated to the VulnDjango directory and scanned that directory with bandit using the command: bandit -r .



To be able to save the output as a json file I ran the command again but outputting the results in a json file:

bandit -r . -f json | tee bandit-output.json



Bandit Security Scan Summary

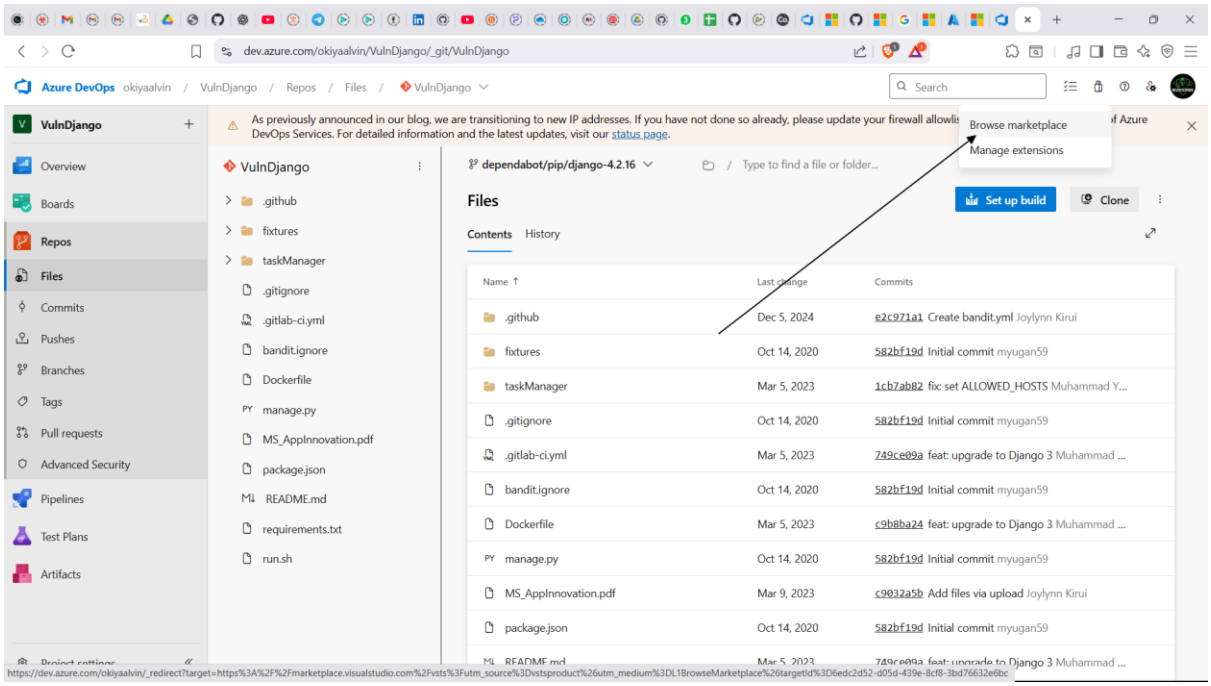
The Bandit static code analysis identified critical security issues within the VulnDjango codebase. The scan flagged a high-severity **command injection risk** due to the use of os.system() with unsanitized input, as well as **hardcoded secrets** (AWS keys, Slack tokens, and Django secret keys) that could expose sensitive credentials if leaked. A **medium-severity SQL injection risk** was detected from direct string-based query construction, alongside other low-severity findings like empty password strings. To mitigate these risks, it is recommended to replace unsafe system calls with subprocess.run(), remove hardcoded secrets by using environment variables or a secrets manager, and adopt parameterized queries or ORM methods to prevent SQL injection attacks.

Exercise 3: Configure the Microsoft Security DevOps Azure DevOps extension

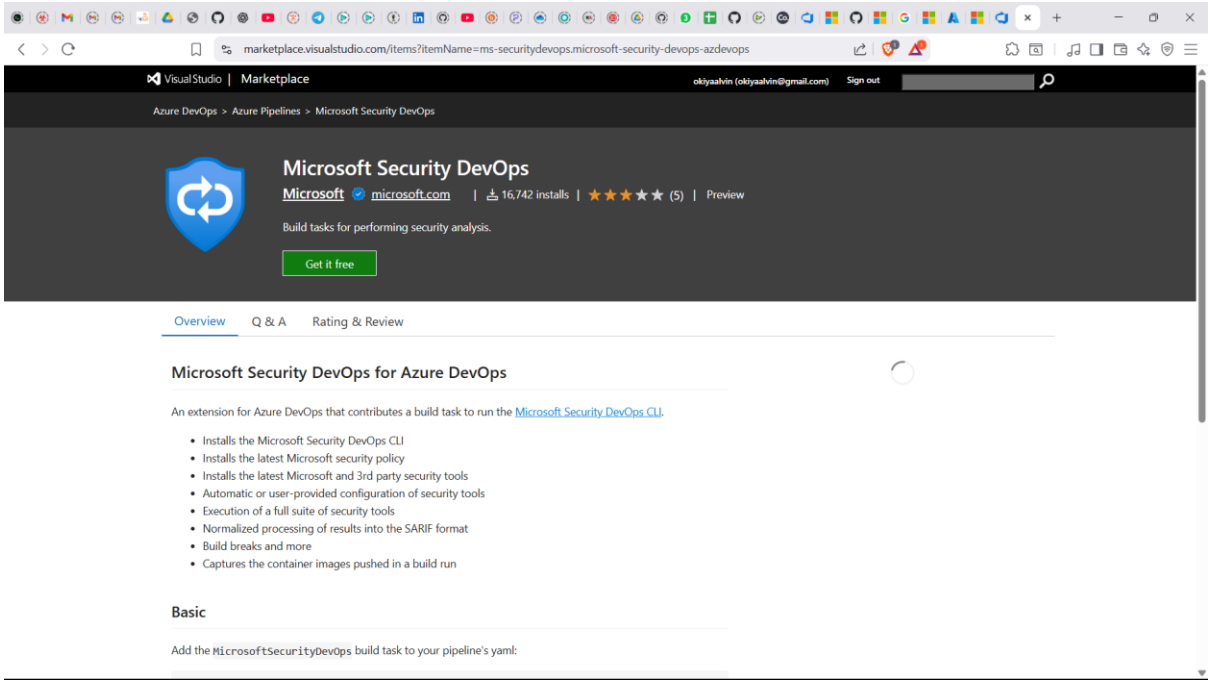
We need to ensure that we have Microsoft Defender for devops deployed in azure devops. The steps below will ensure that this is done and the extension is installed from the marketplace.

Note: Admin privileges to the Azure DevOps organization are required to install the extension.

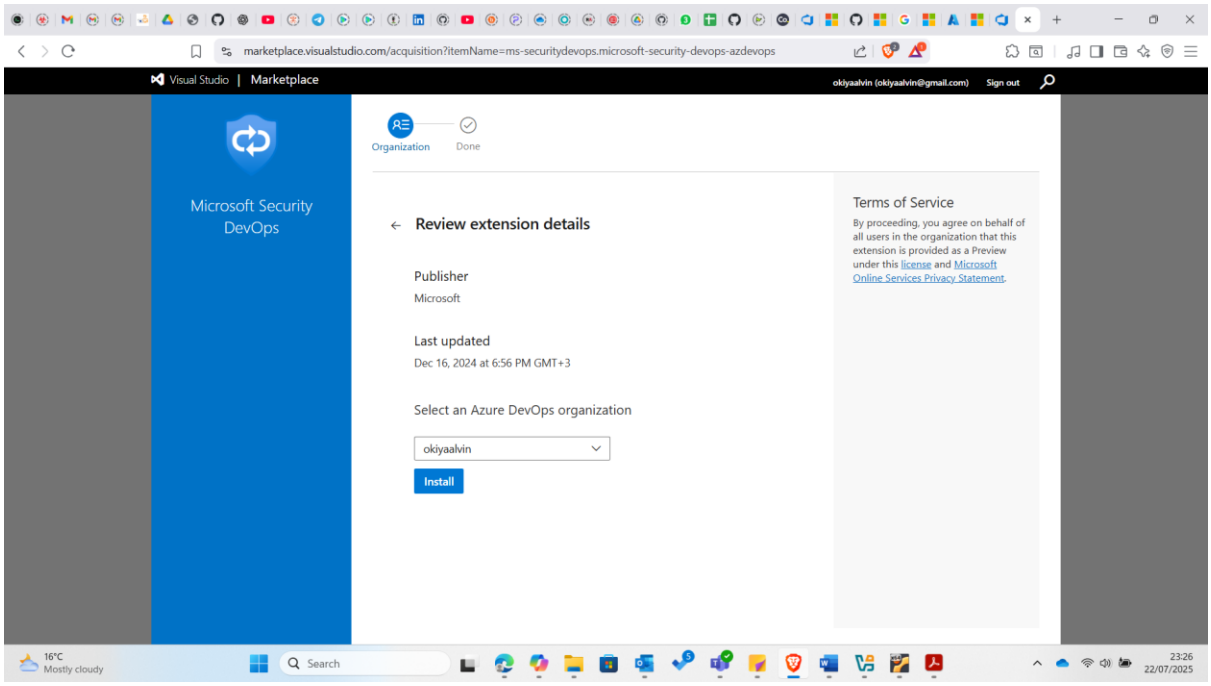
I first activated Microsoft Security DevOps extension –on my Azure DevOps portal with the**VulnDjango**project open, I clicked on the marketplace icon >**Browse Marketplace**



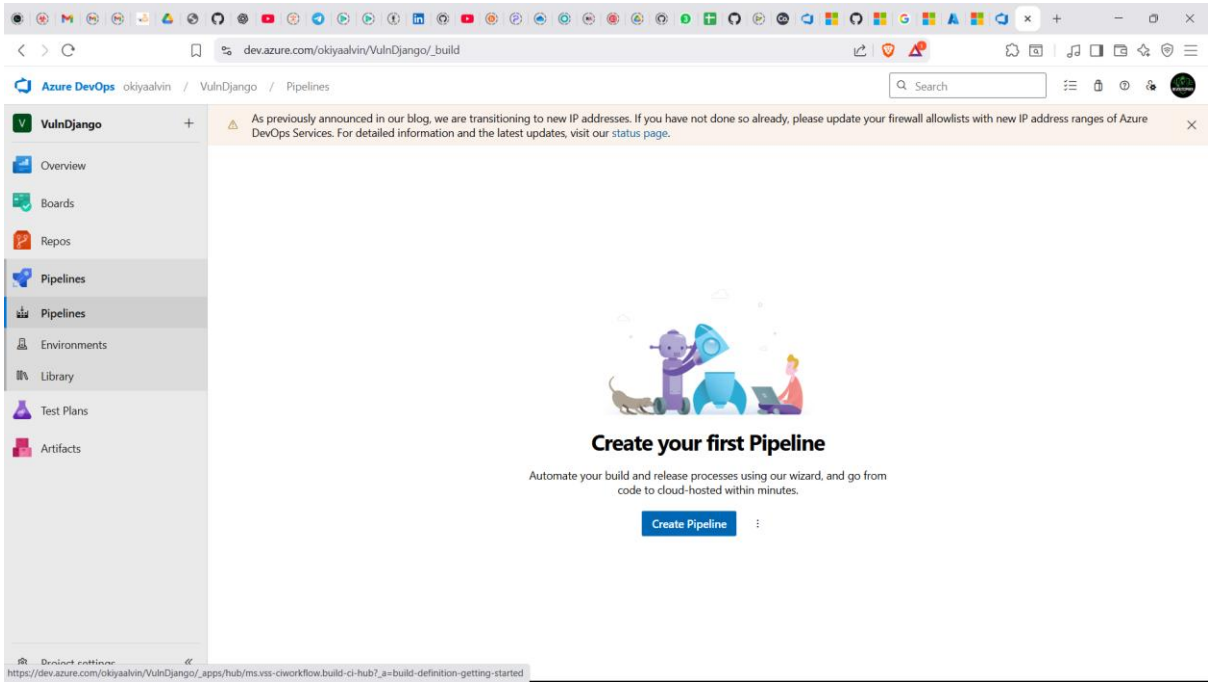
On the Marketplace, I searched for **Microsoft Security DevOps** and opened it.



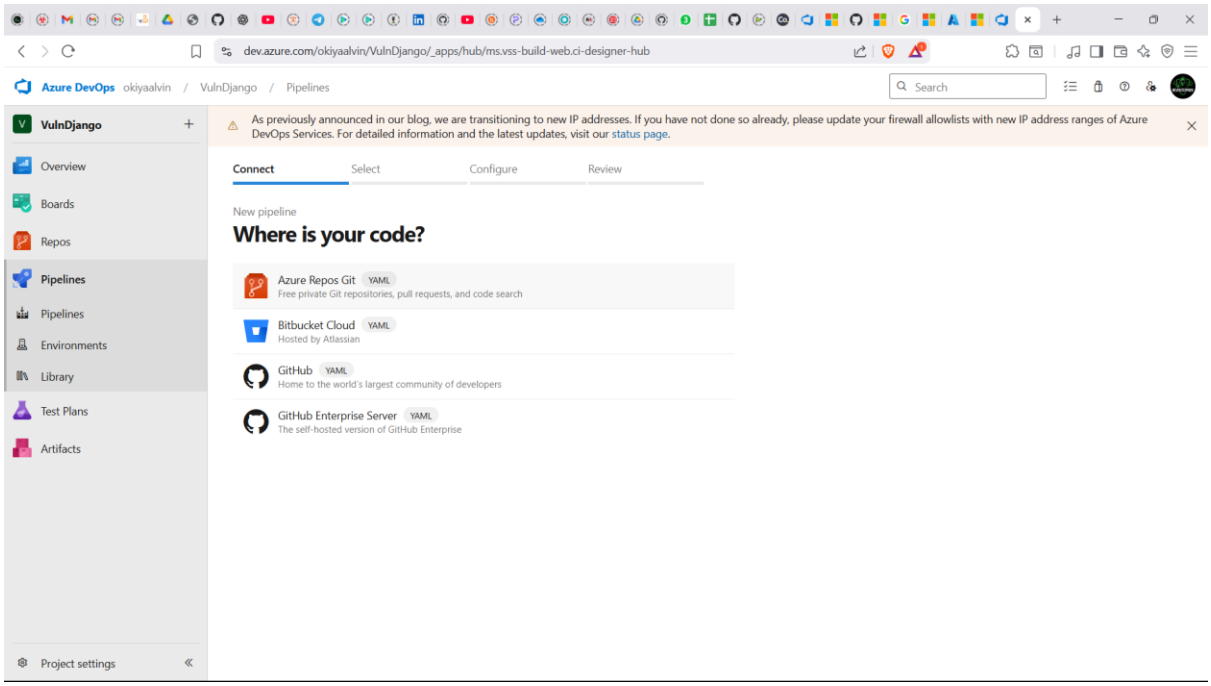
Then I clicked Get it for free and on the next page, I selected the desired Azure DevOps organization which in my case was okiyaalvin and then clicked Install. Next was to Proceed to organization once installed.

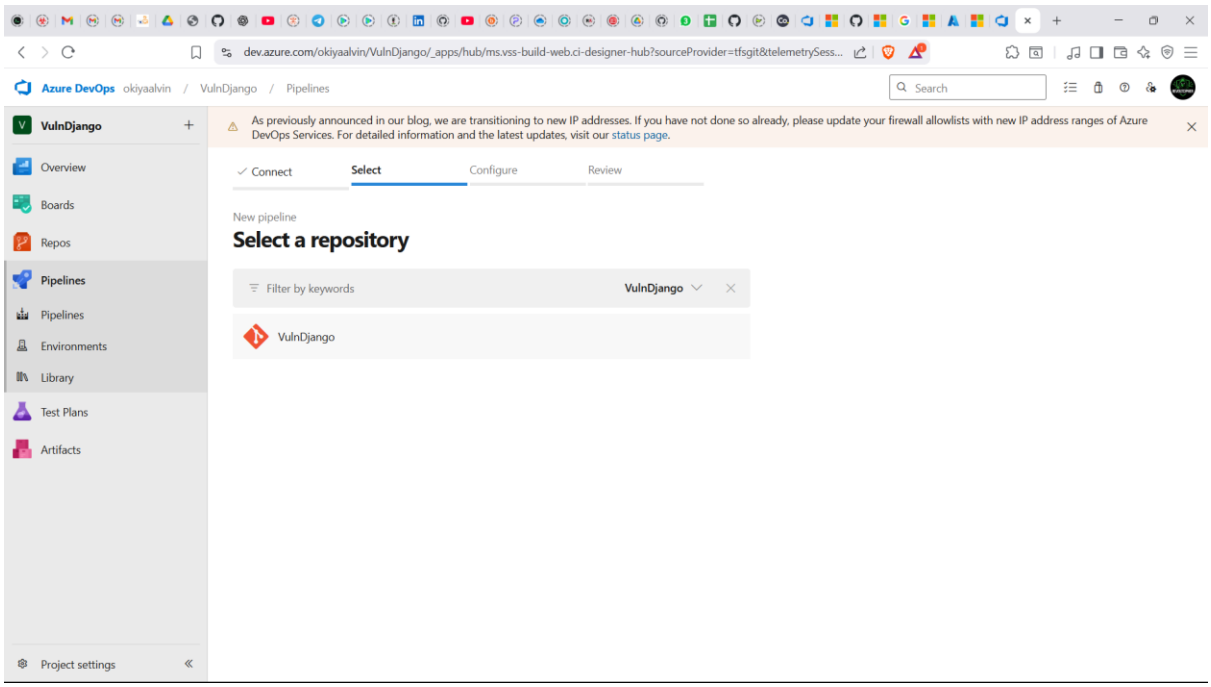


Next was to Navigate to my VulnDjangoproject, then Pipelines and Click New pipeline.

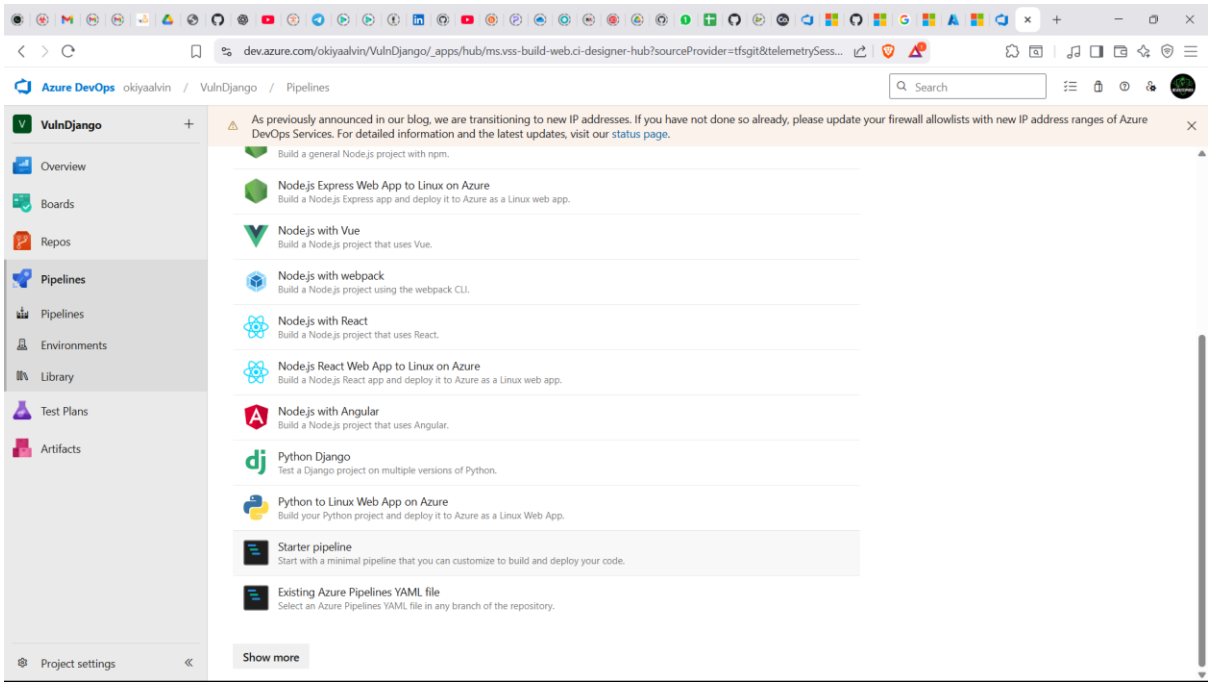


On theWhere is your code? window, I selected Azure Repos Git (YAML)and selected theVulnDjangorepository.





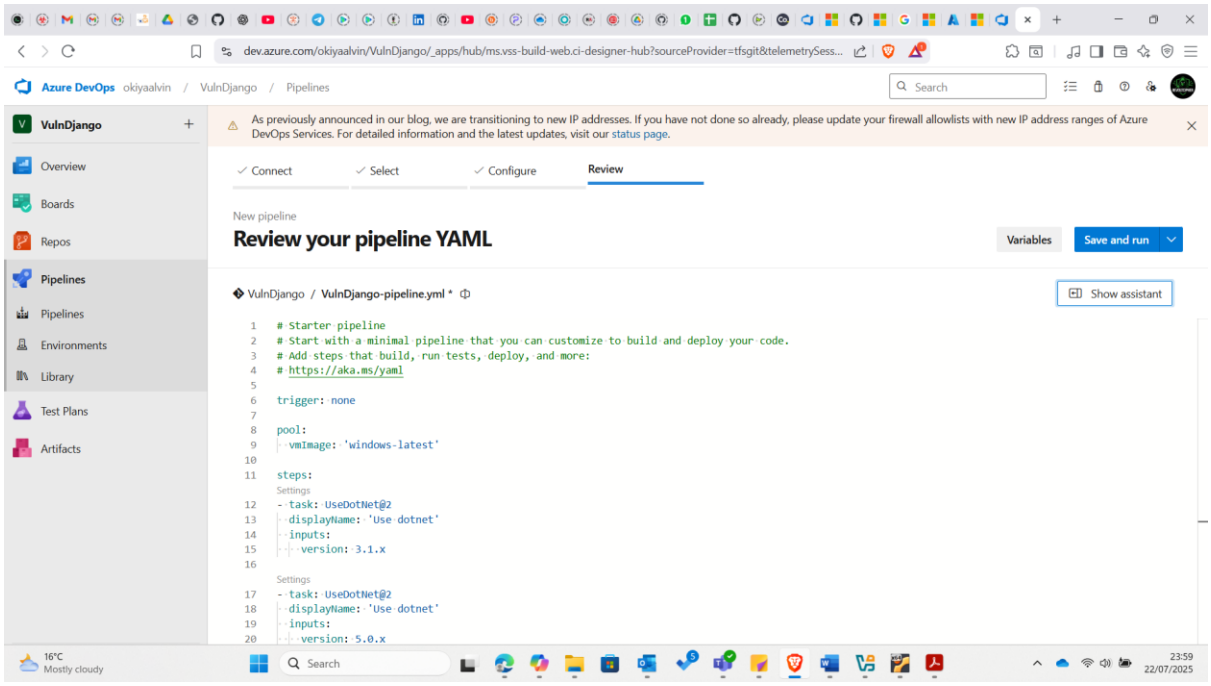
I Selected the starter pipeline as shown:



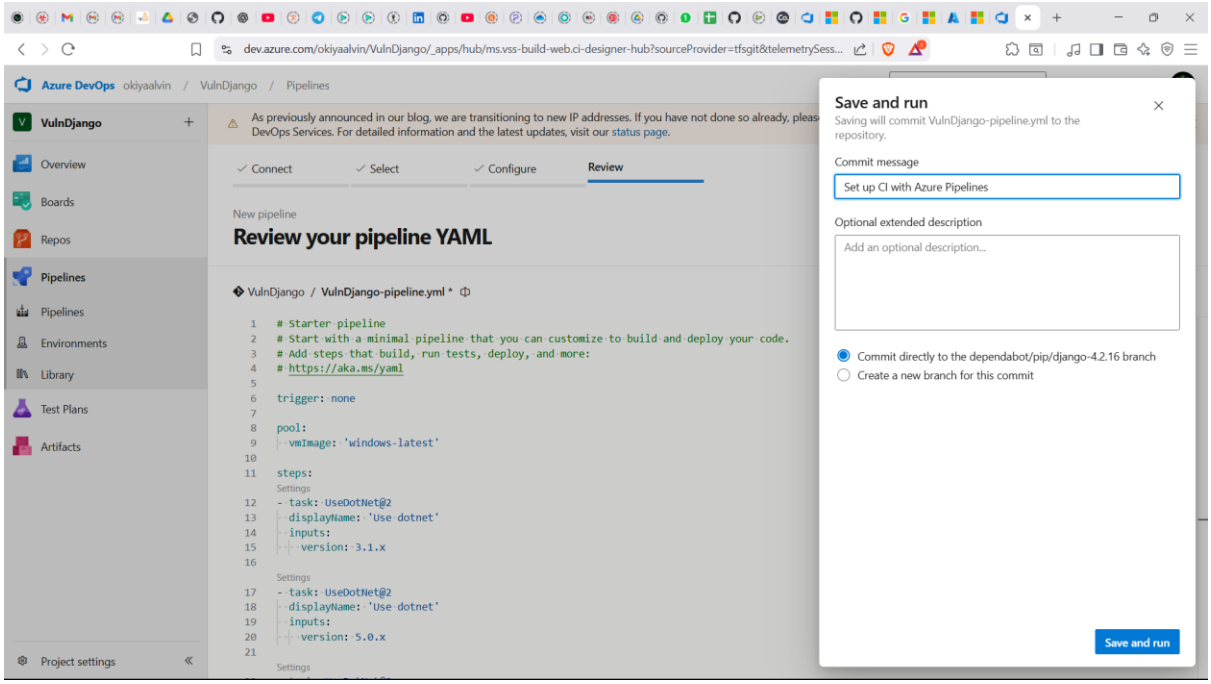
Then replaced its content with the following scripts as in into the yaml file.

```yaml
# Starter pipeline
# Start with a minimal pipeline that you can customize to build and deploy your code.
# Add steps that build, run tests, deploy, and more:
# https://aka.ms/yaml
trigger: none
pool:
  vmImage: 'windows-latest'
steps:
- task: UseDotNet@2
  displayName: 'Use dotnet'
  inputs:
    version: 3.1.x
- task: UseDotNet@2
  displayName: 'Use dotnet'
  inputs:
    version: 5.0.x
- task: UseDotNet@2
  displayName: 'Use dotnet'
  inputs:
    version: 6.0.x
- task: MicrosoftSecurityDevOps@1
  displayName: 'Microsoft Security DevOps'
```

And renamed it VulnDjango-pipeline.yaml

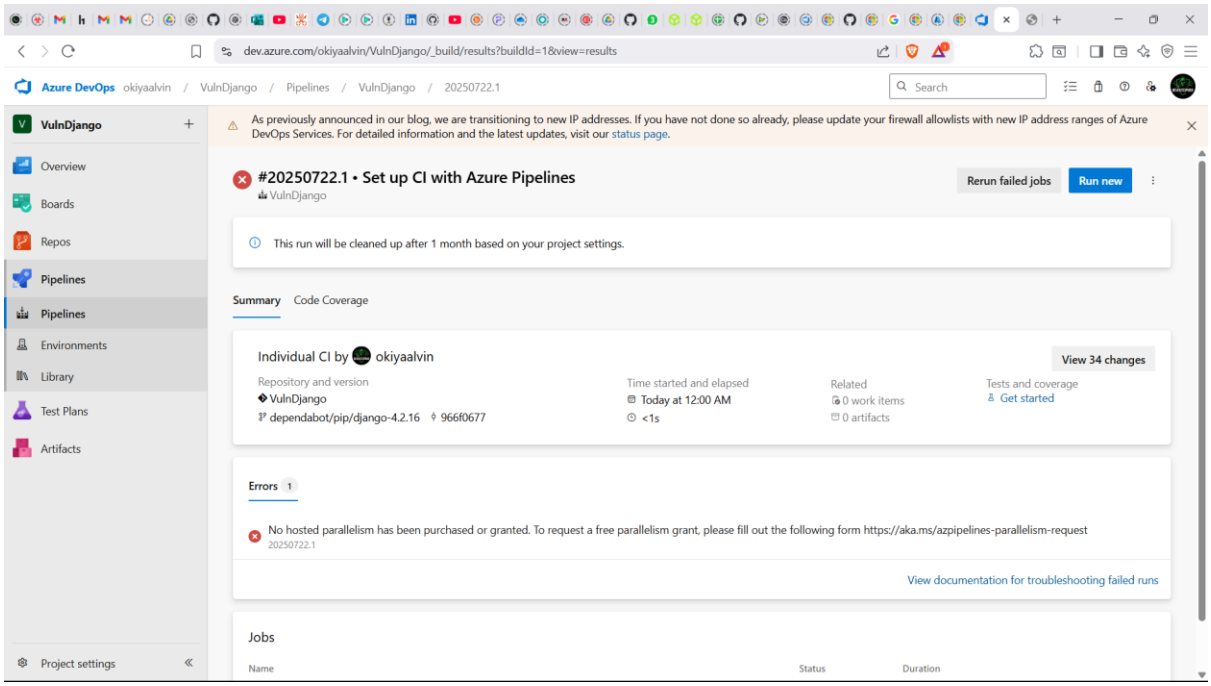


I then clicked on save and run



Error 1: No hosted parallelism has been purchased or granted

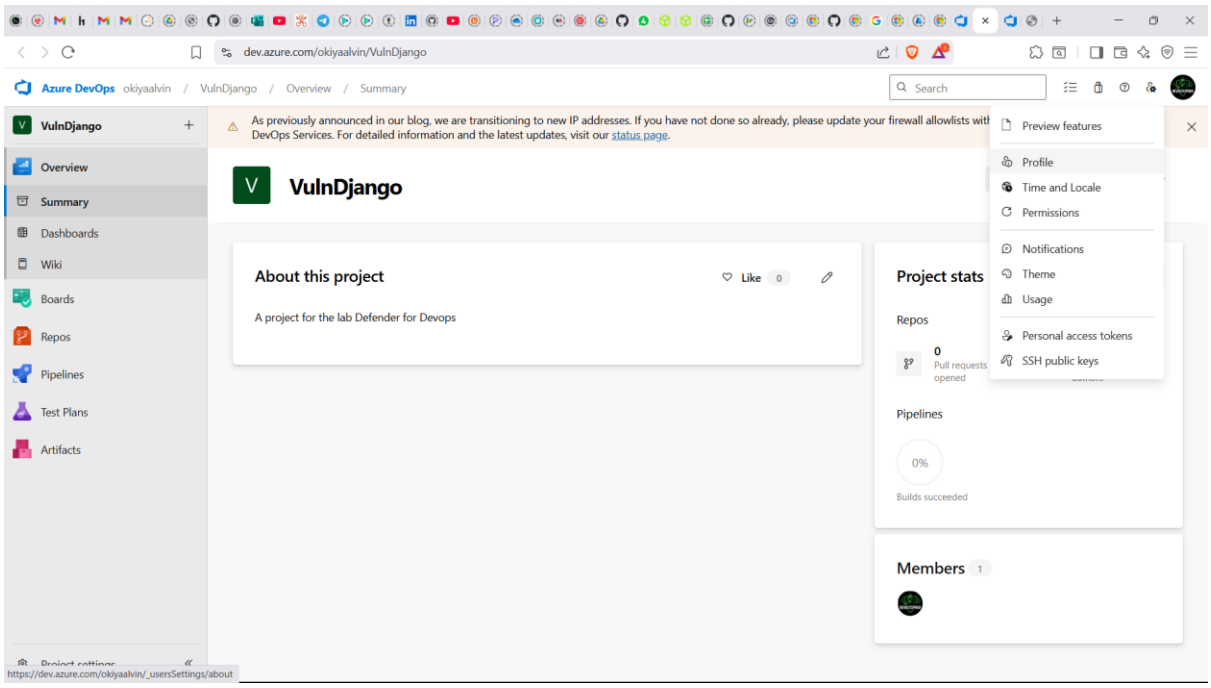
However it failed to run with an error of No hosted parallelism has been purchased or granted.



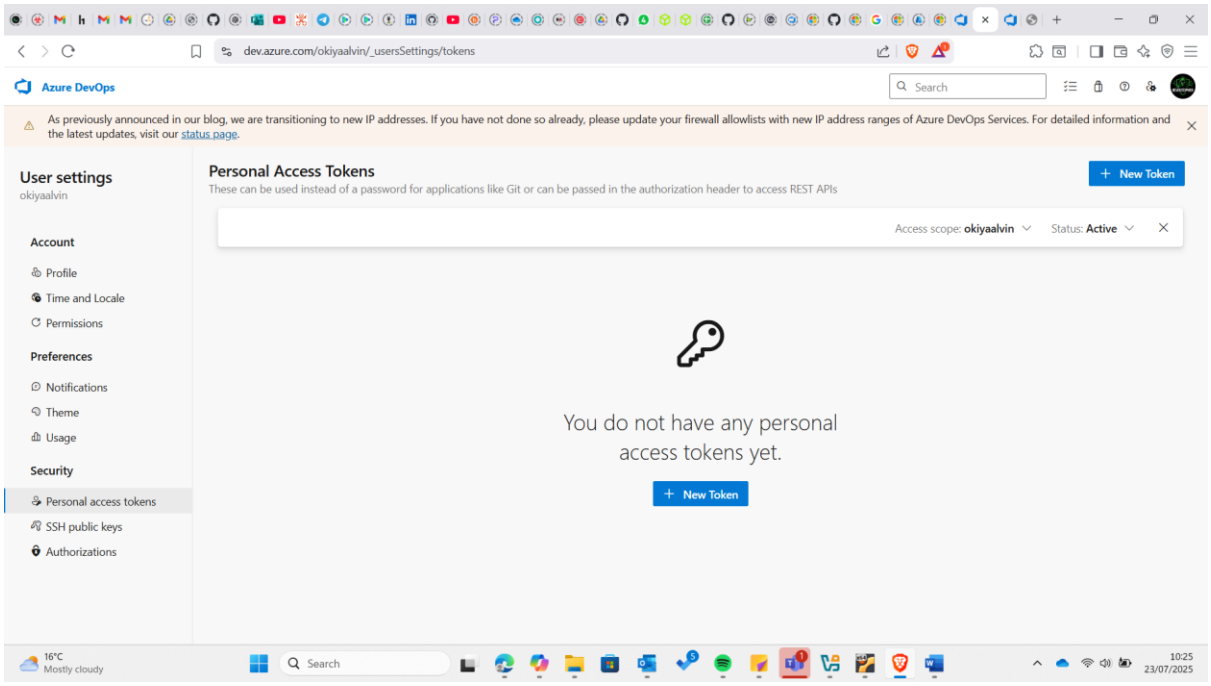
Troubleshooting

Due to this I opted to use a self hosted agent, which was my kali linux vbox.

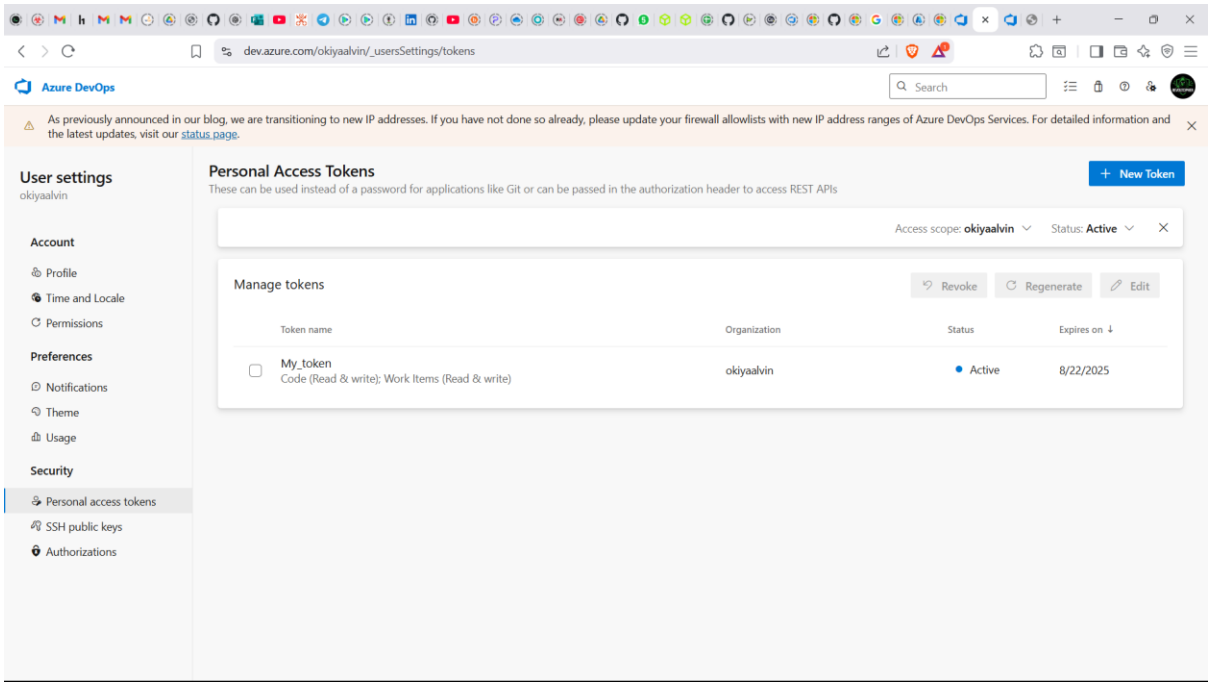
To do this I first created a personal access token (PAT) by going to the profile:



Then security > Personal Access Tokens

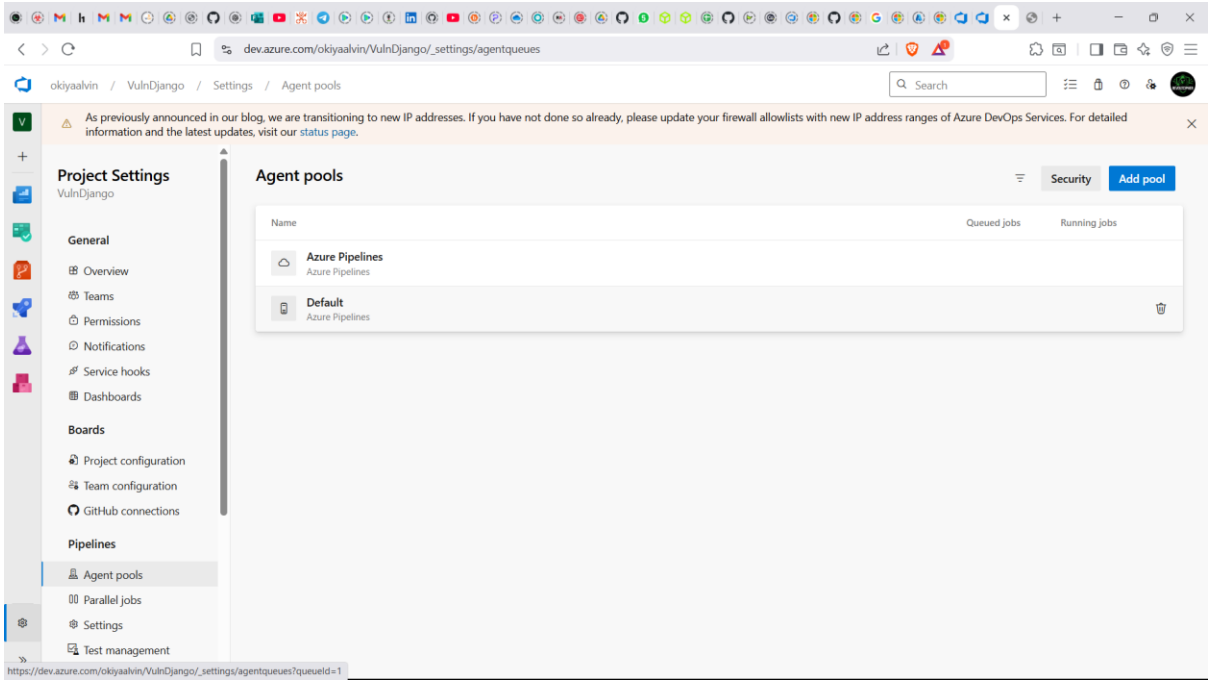


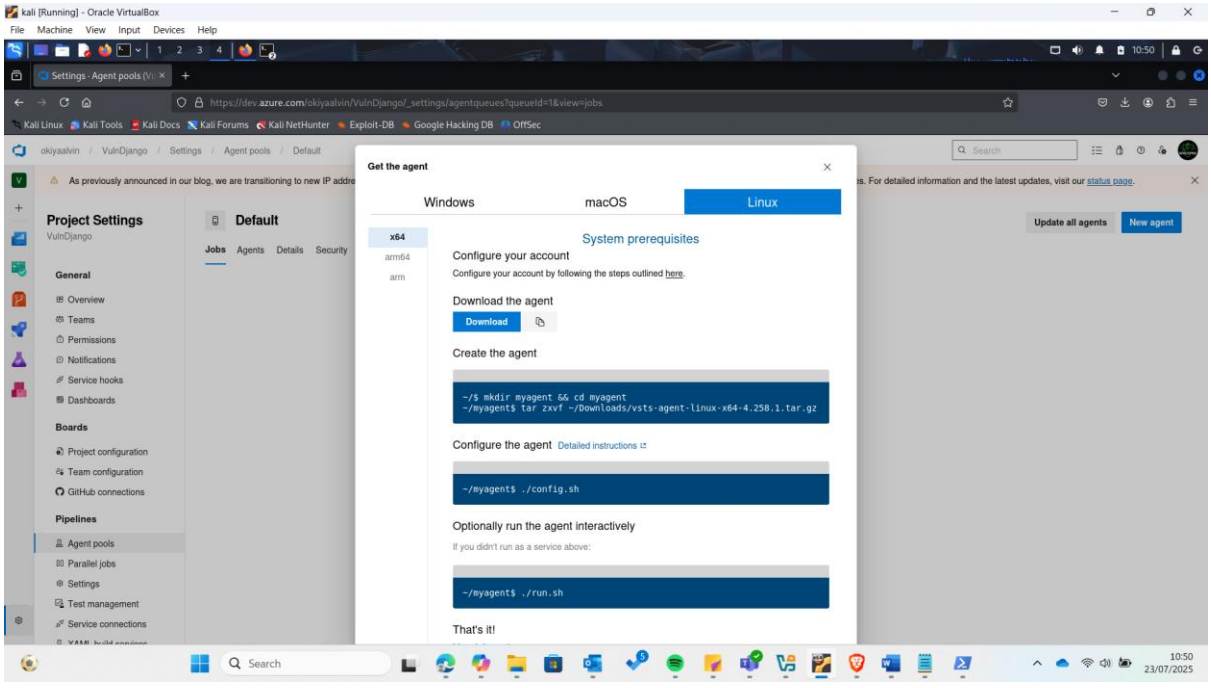
I then created a token, I only checked three sections during its configuration: on code and work items, I checked read and write and on Agent pools I checked read and manage. Then copied the Personal Access Token and saved on a txt file:



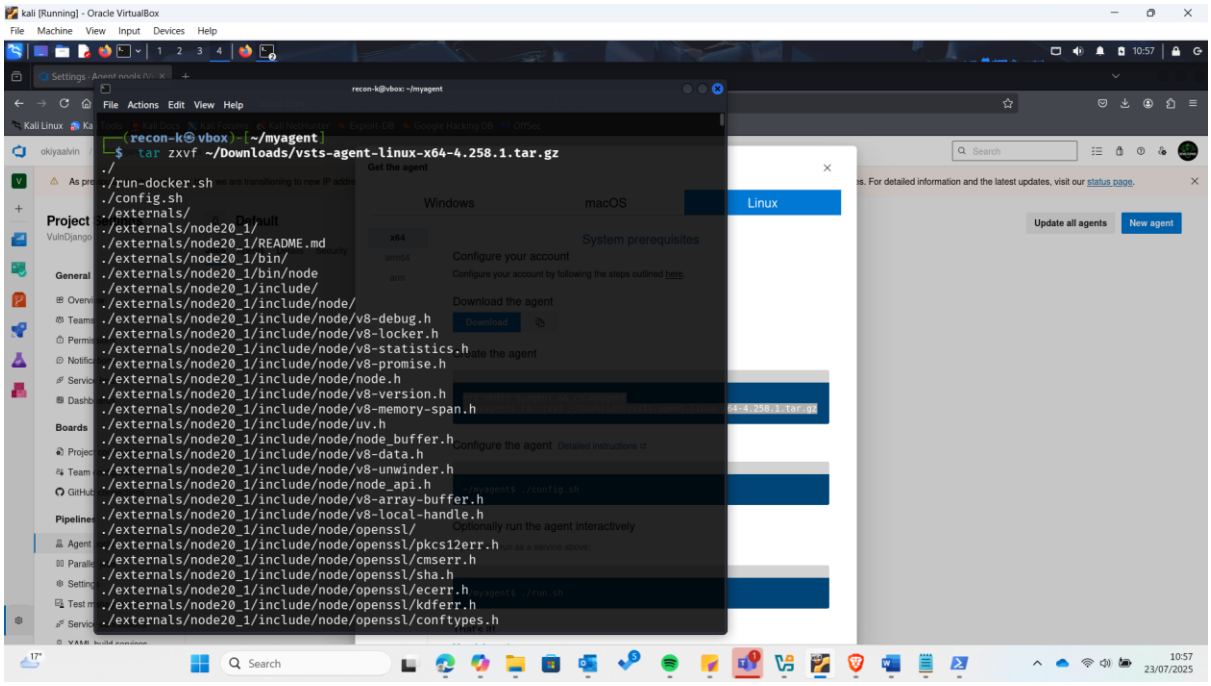
Then I downloaded and configured the agent;

1. Navigated to my project in Azure DevOps
2. Went to **Project Settings → Agent Pools → Default**
3. Clicked **New Agent** → Choose your OS → Download agent package

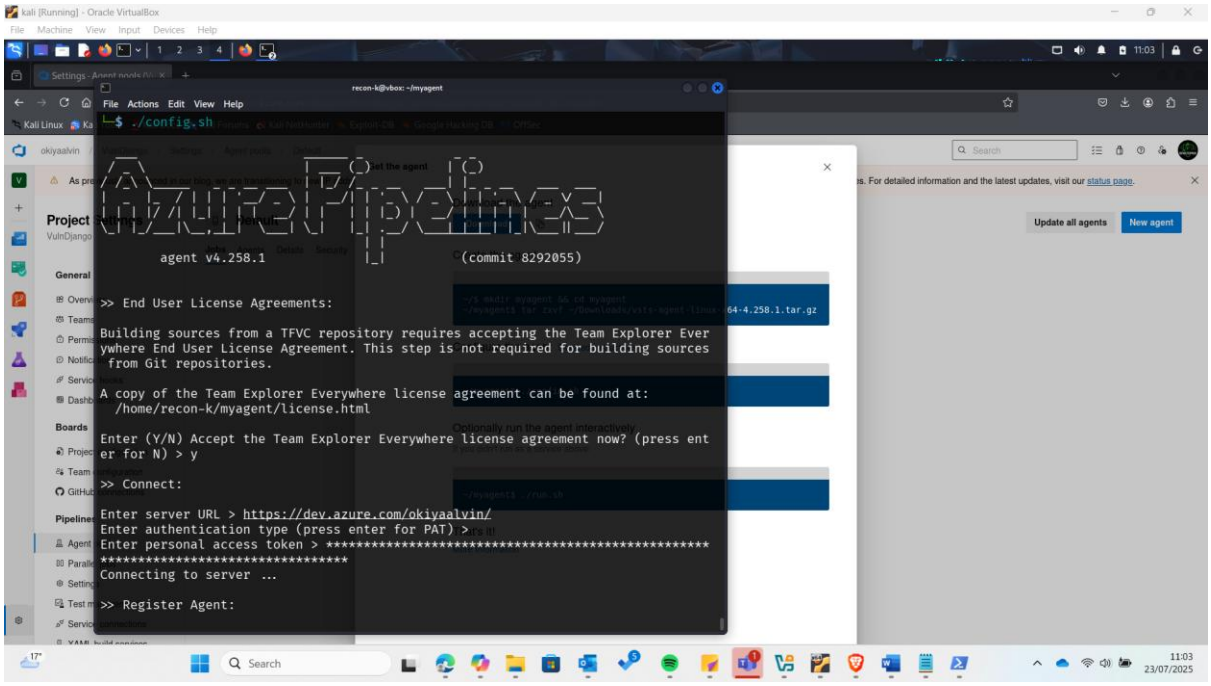


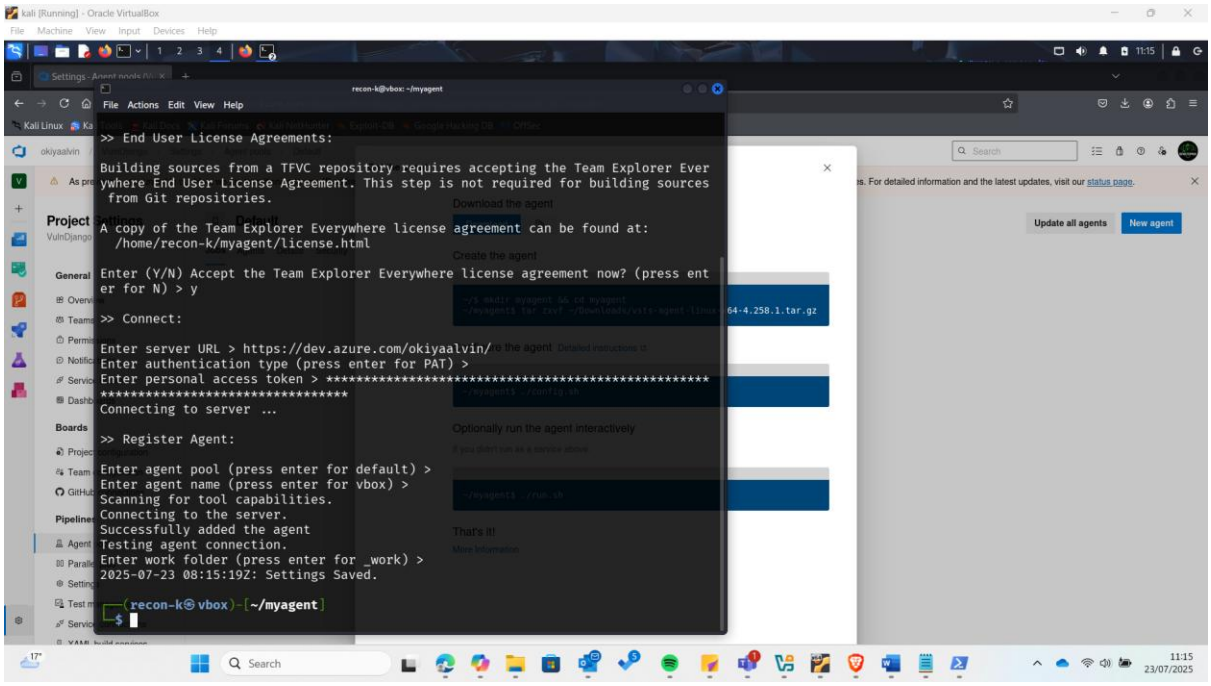


Then extracted the .gz file:

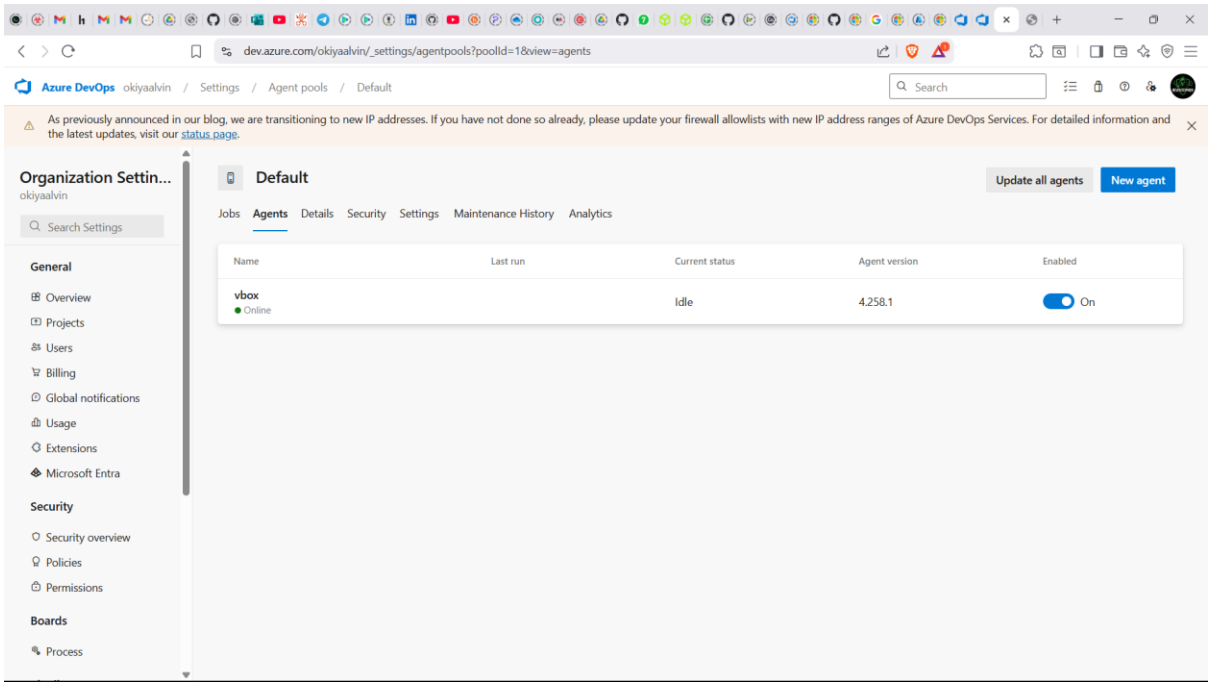


Then configured the agent by running ./config.sh and entering the server url as my azure devops organization url and also entered the PAT:

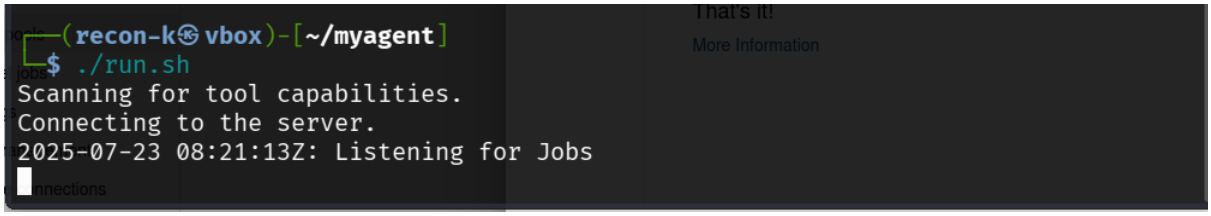




Confirming on the portal, the agent was reflecting as registered:



I then ran the command below to test connection:



Then changed the pipeline YAML file on the azure devops portal to indicate that it will now use the self-hosted agent instead of Microsoft-hosted ones.

To do this **I replaced the line**:

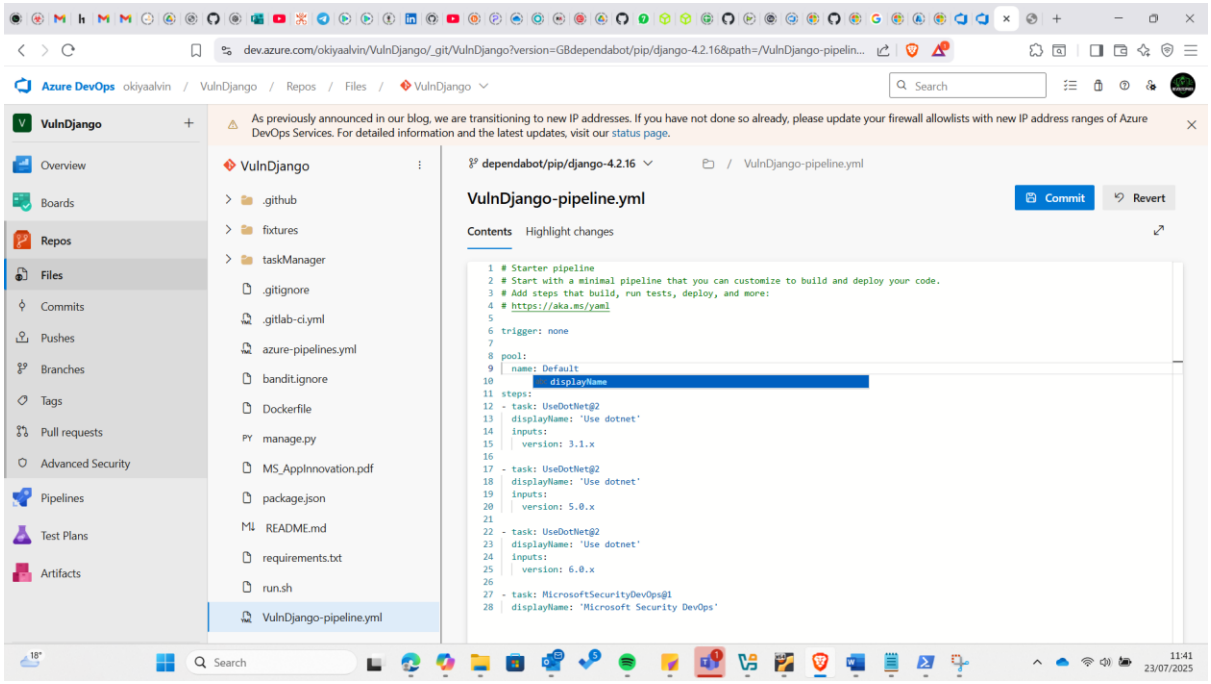
pool:

vmImage: 'windows-latest'

with this line

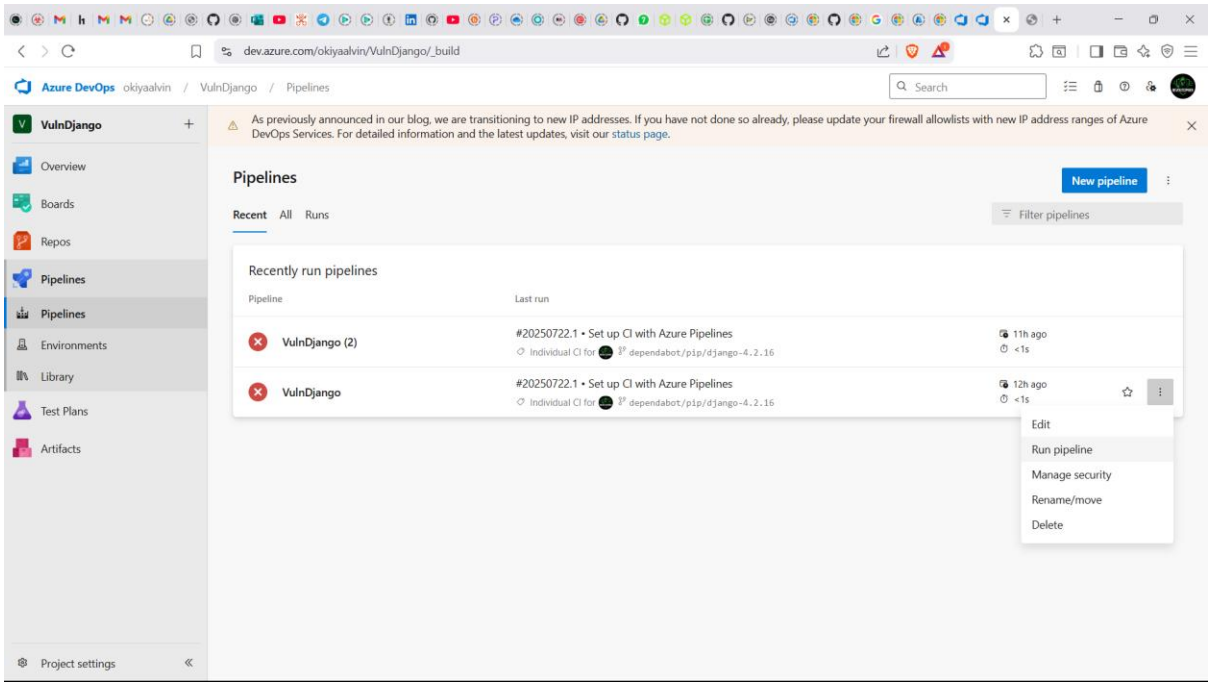
pool:

name: Default

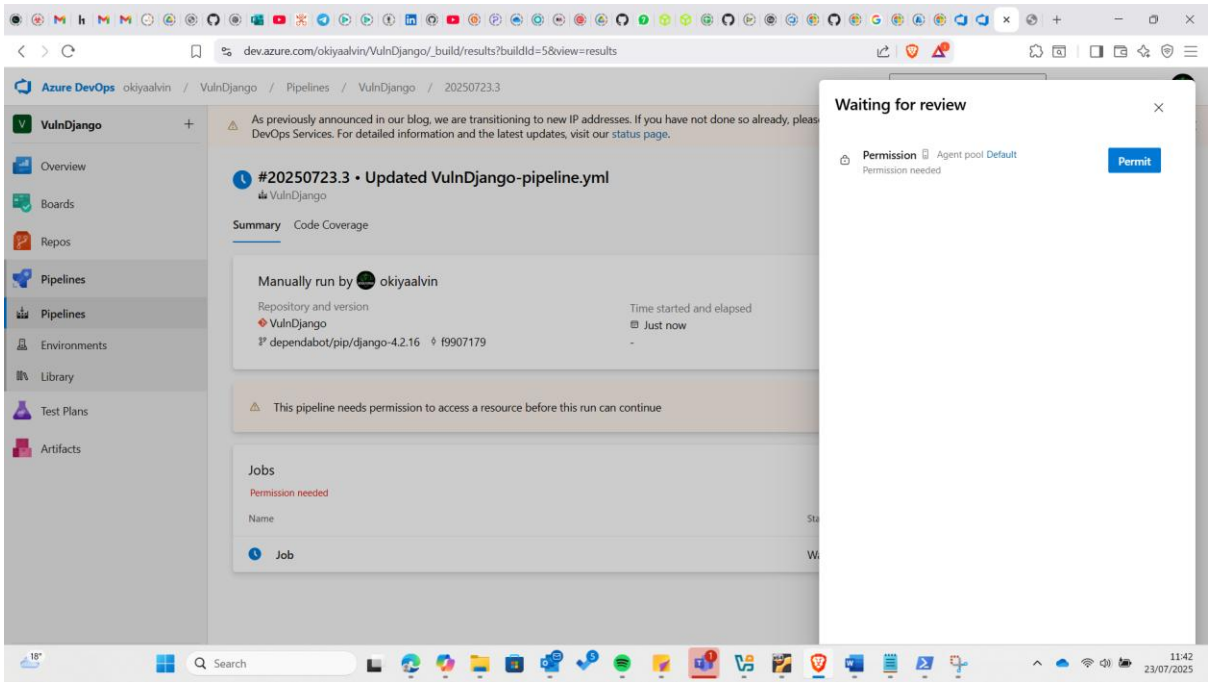


Then I pressed the blue button commit.

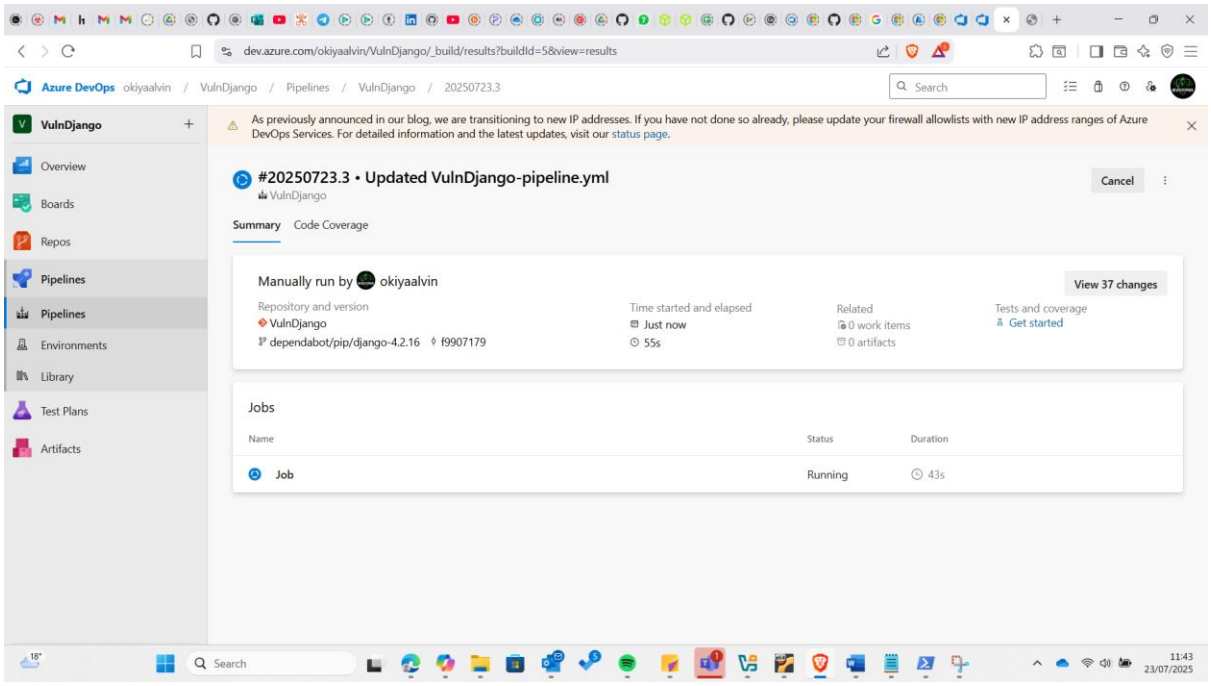
I then navigated to the pipelines section and ran the pipeline again:



When the pipeline requested for permission, I granted:



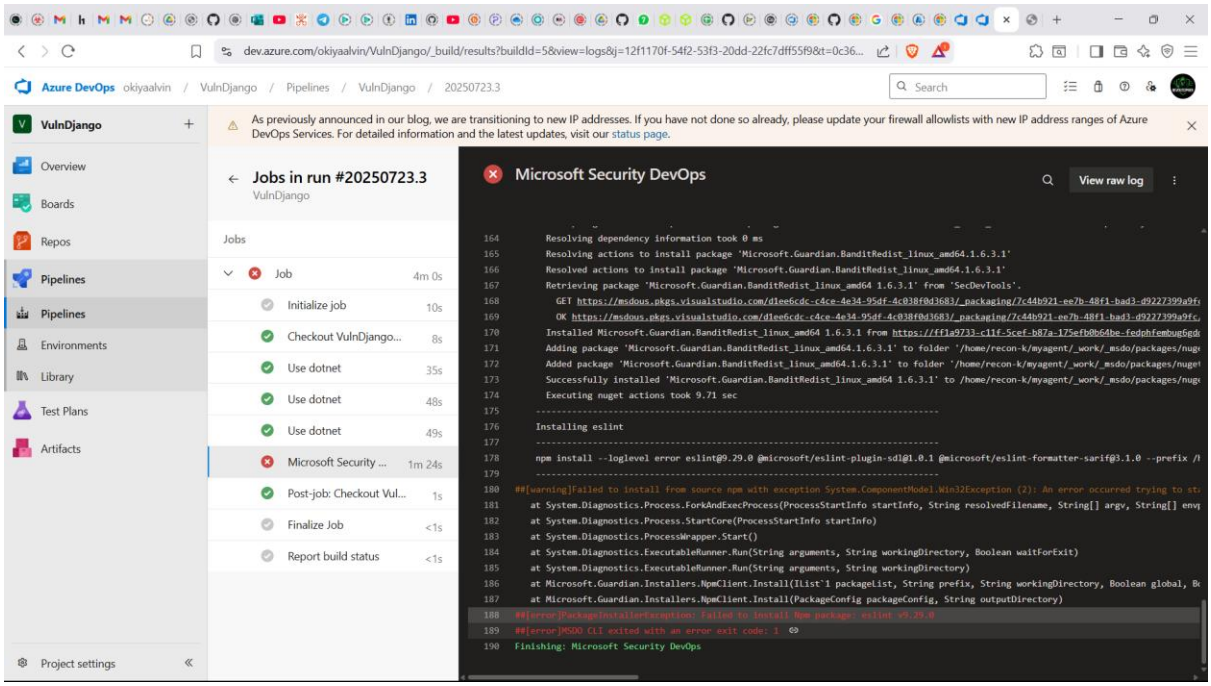
Having done this the pipeline started running



I had now successfully solved that blocker.

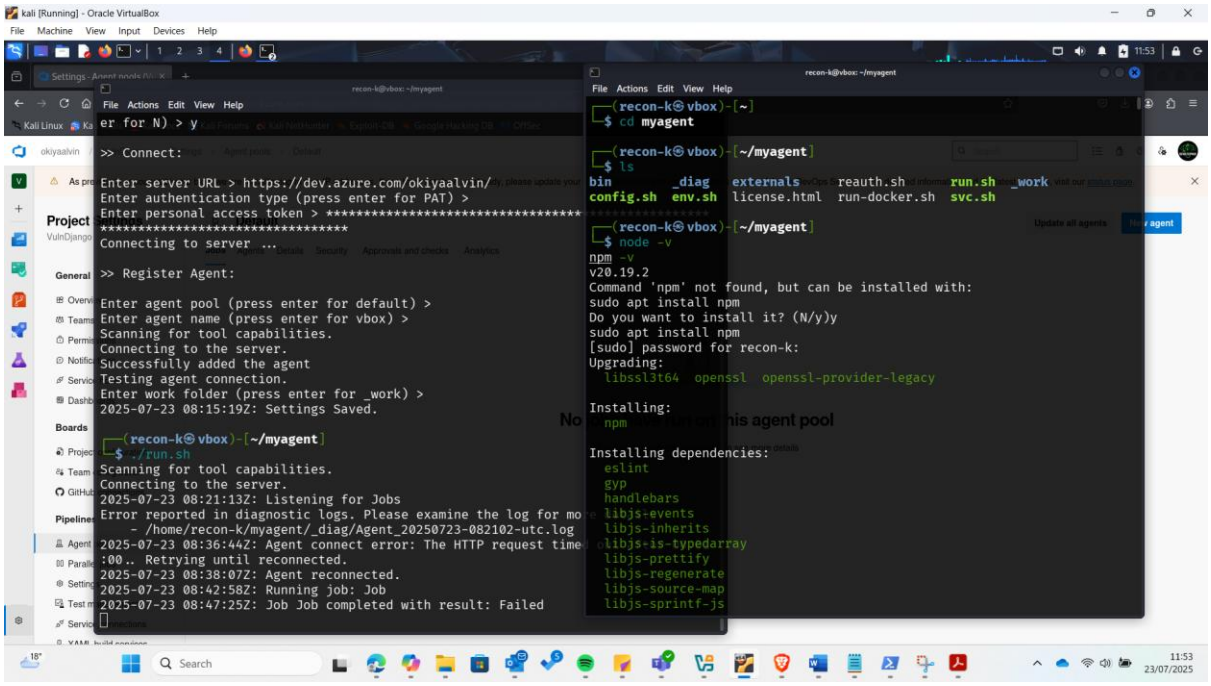
Error 2: Failure to install dependancies

However, there was an error of some dependancies that failed to install on my agent due to some needed privileges

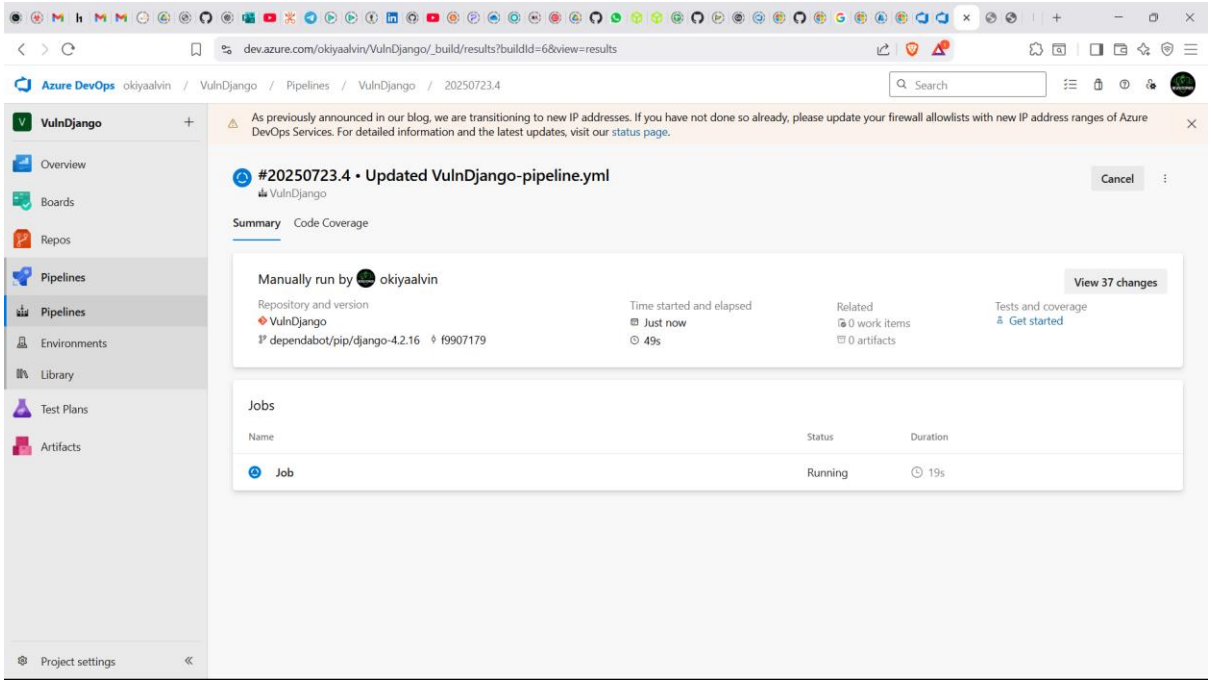


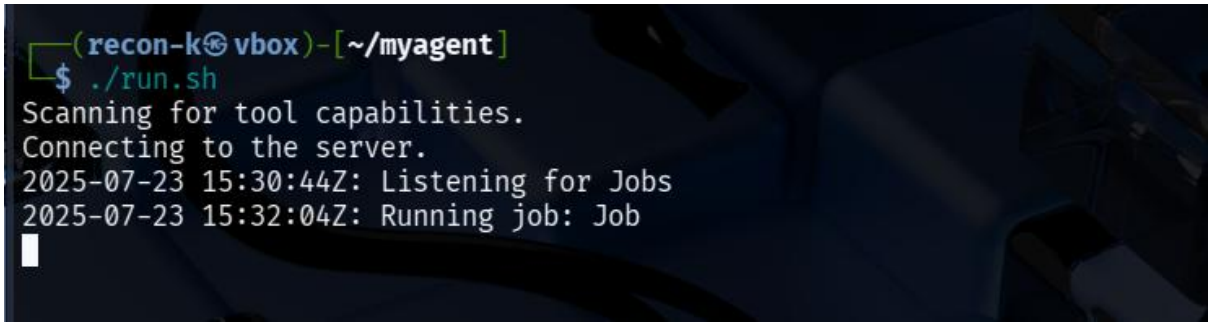
Solution

I decided to install them manually:

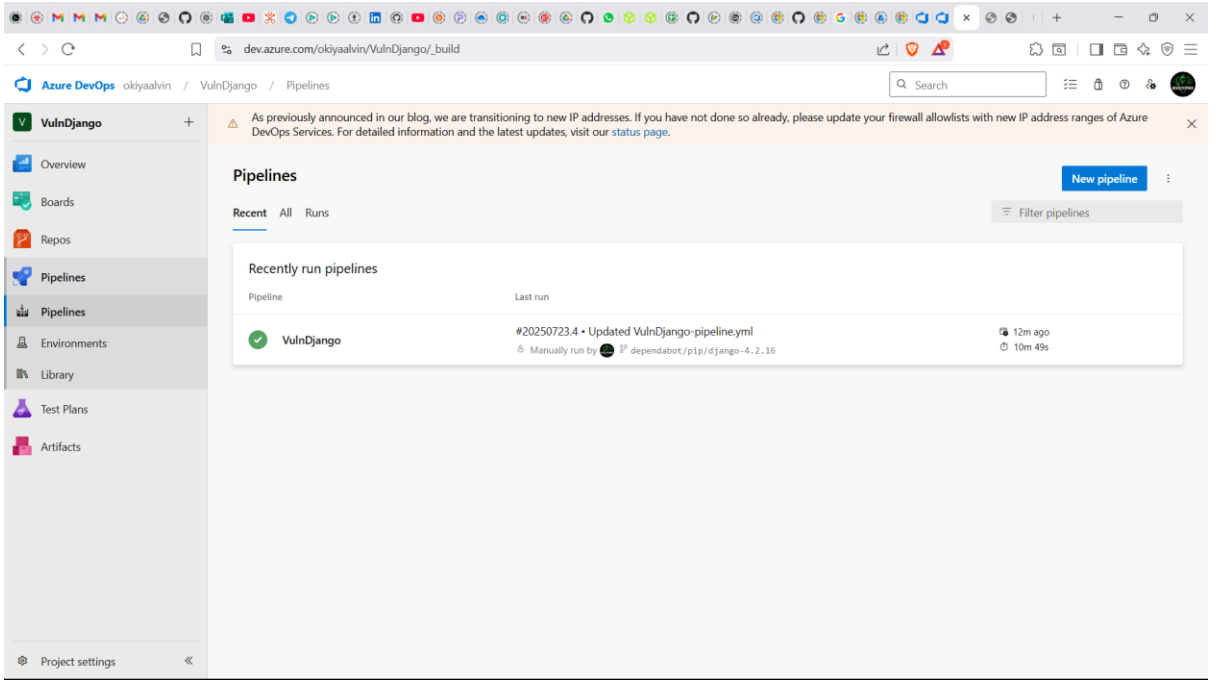


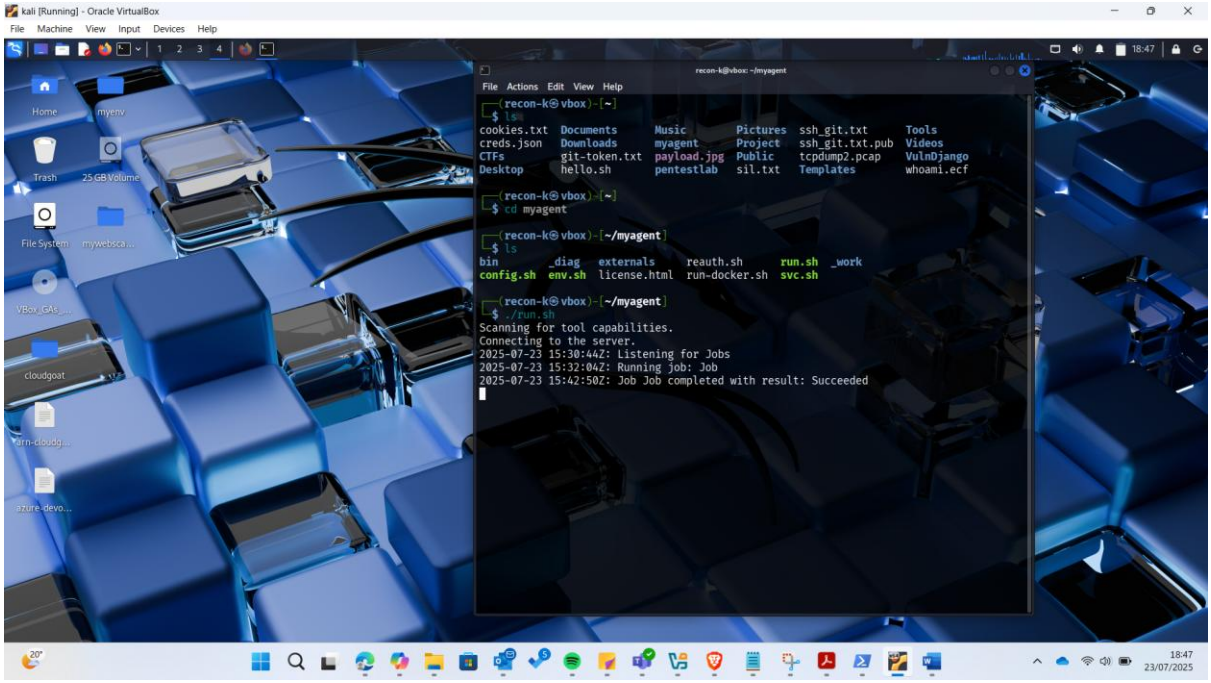
After Installing them I ran the pipeline again:





Finally after more than 10 minutes it was a success:

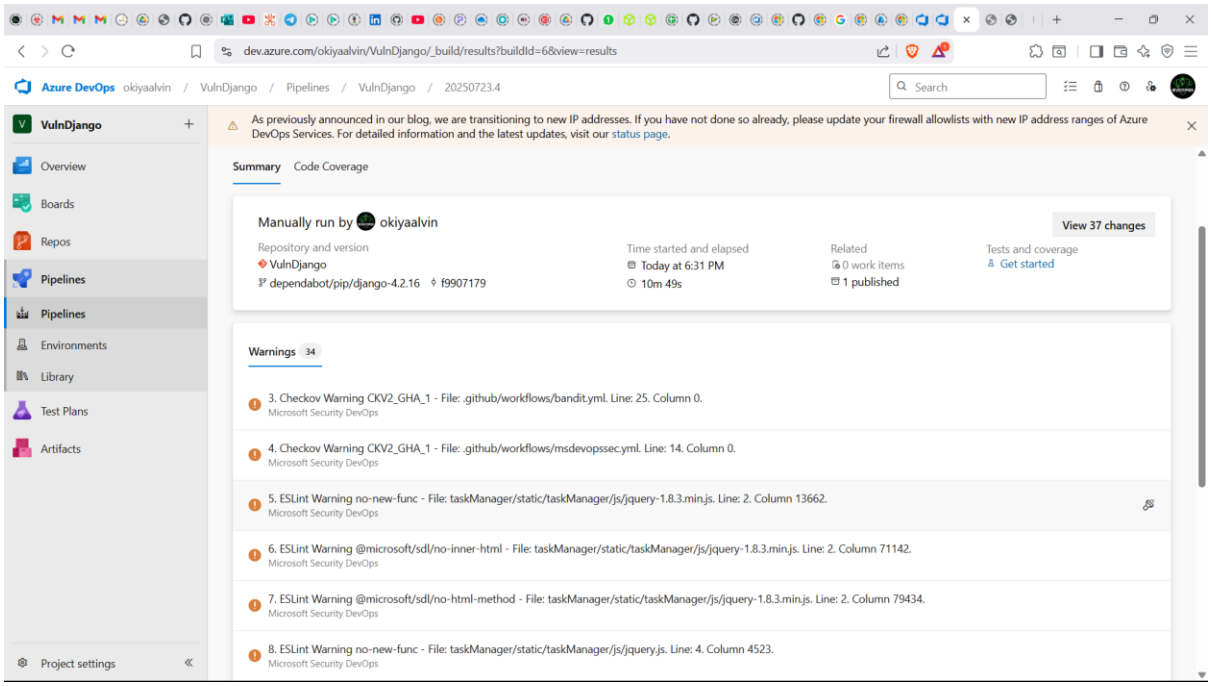




Analysing the Scan

I opened the pipeline to check whether the Microsoft Defender for DevOps (MSDO) ran successfully.

From the pipeline logs and recent status revealed by the following screenshot below:



What's Confirmed

- The MSDO CLI task ran — that means Defender for DevOps kicked in.

- It produced security and quality warnings via ESLint and Checkov, which MSDO integrates.

- The pipeline reached the end without error, indicating the scan didn't crash or get skipped.

How MSDO Works Behind the Scenes

MSDO pulls in security tools like:

- ESLint for JavaScript/TypeScript code quality and logic bugs

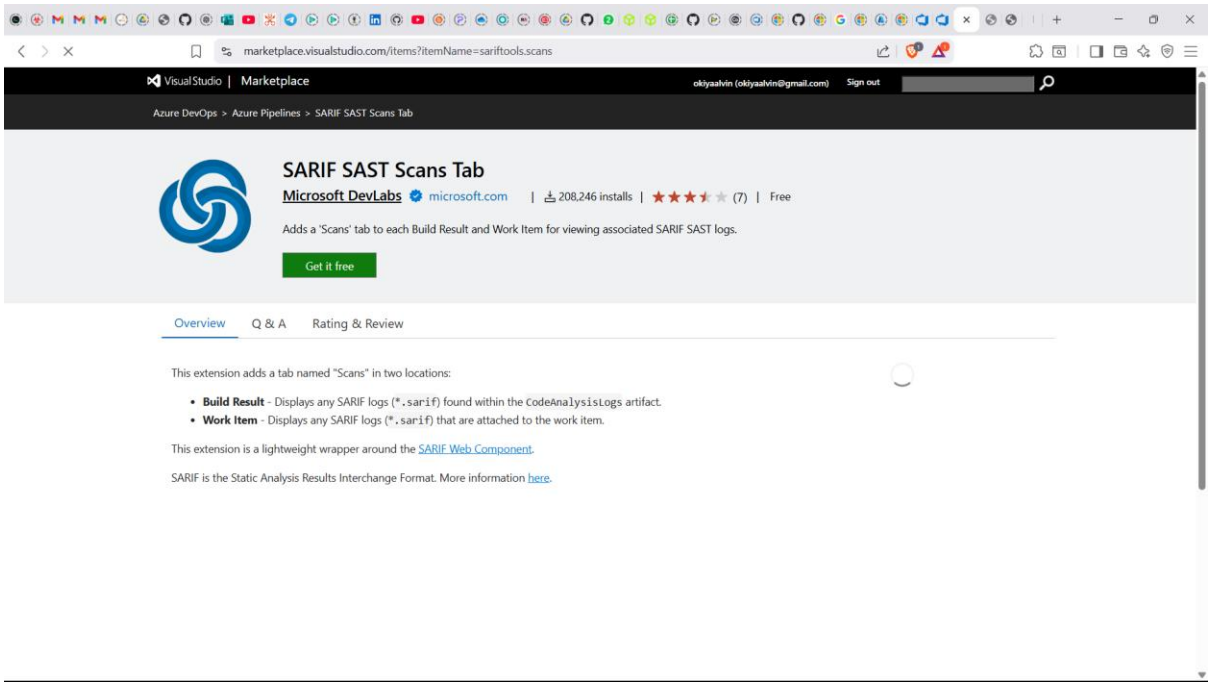
- Checkov for IaC security misconfigs (Terraform, YAML, Dockerfiles)

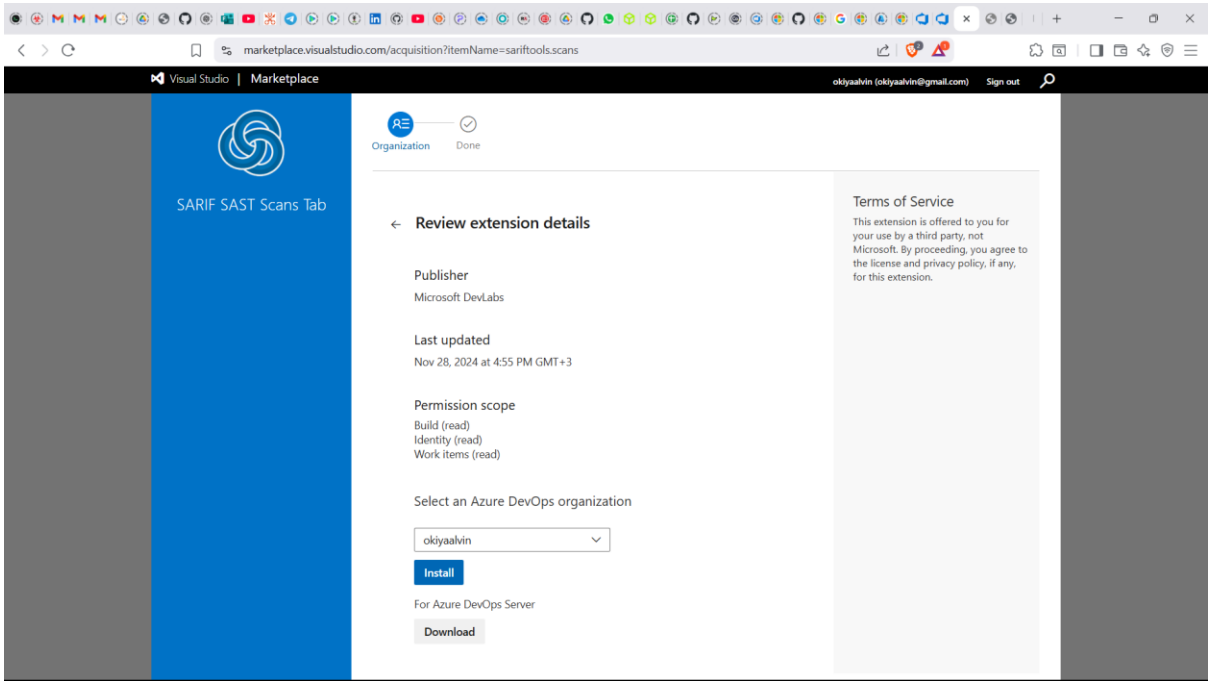
- Optionally, CredScan, BinSkim, and others for deeper code inspection

It aggregates results into formats like SARIF, and those can be uploaded to GitHub Advanced Security or Microsoft Defender for Cloud if integrated.

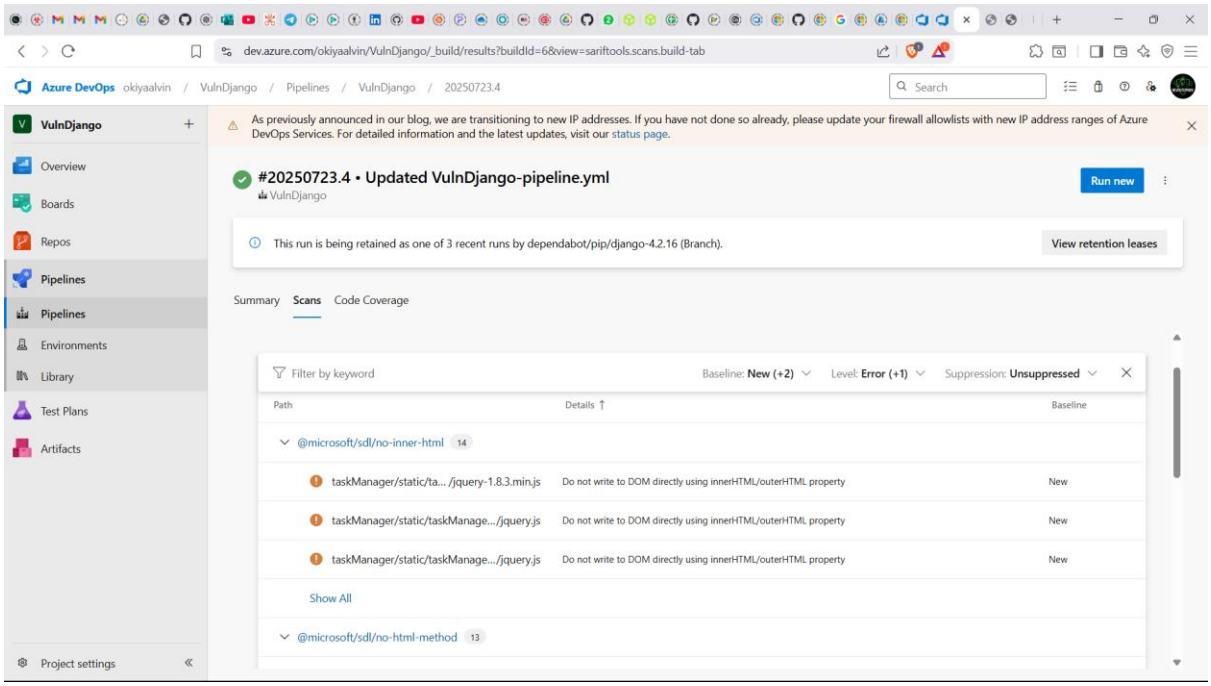
Installing SARIF SAST Scans Tab extension

In order to ensure that the generated analysis results are displayed automatically under the results tab, I installed the SARIF SAST Scans Tab extension on the Azure DevOps organization.





Having installed that, the scan section is now available next to the summary and it shows the scan results of MSDO during running of the pipeline



Exercise 4: Configure the Microsoft Security DevOps GitHub actions

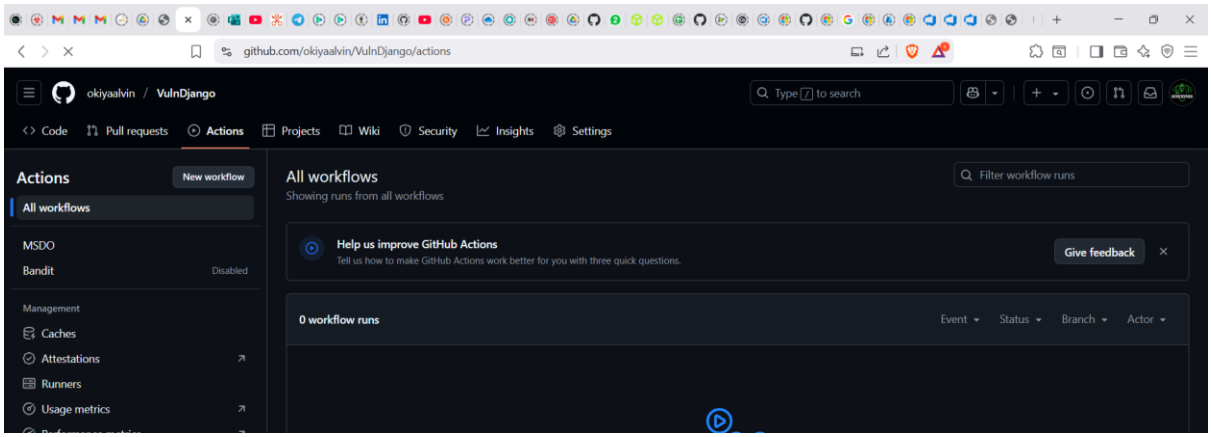
Next exercise was to Configure the Microsoft Security DevOps GitHub actions where I followed the following steps:

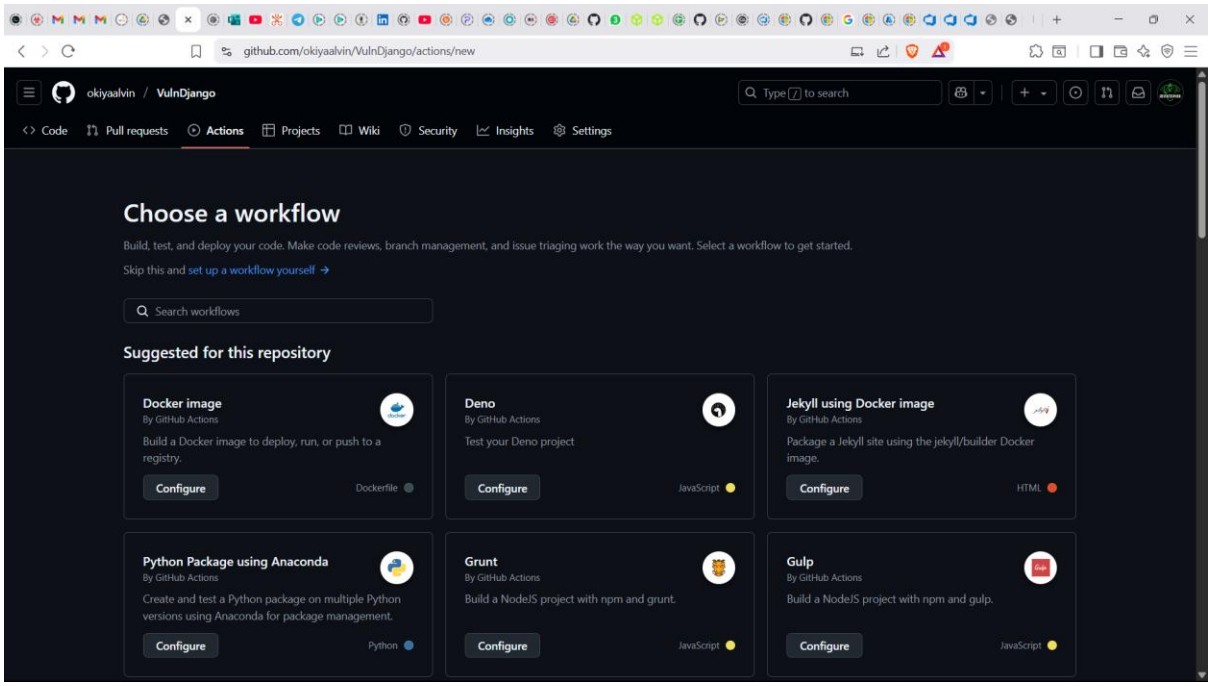
1.Navigate to your VulnDjango GitHub repo.

2.SelectActions

3.Select**New workflow**.

4.On the Get started with GitHub Actions page, select **set up a workflow yourself**

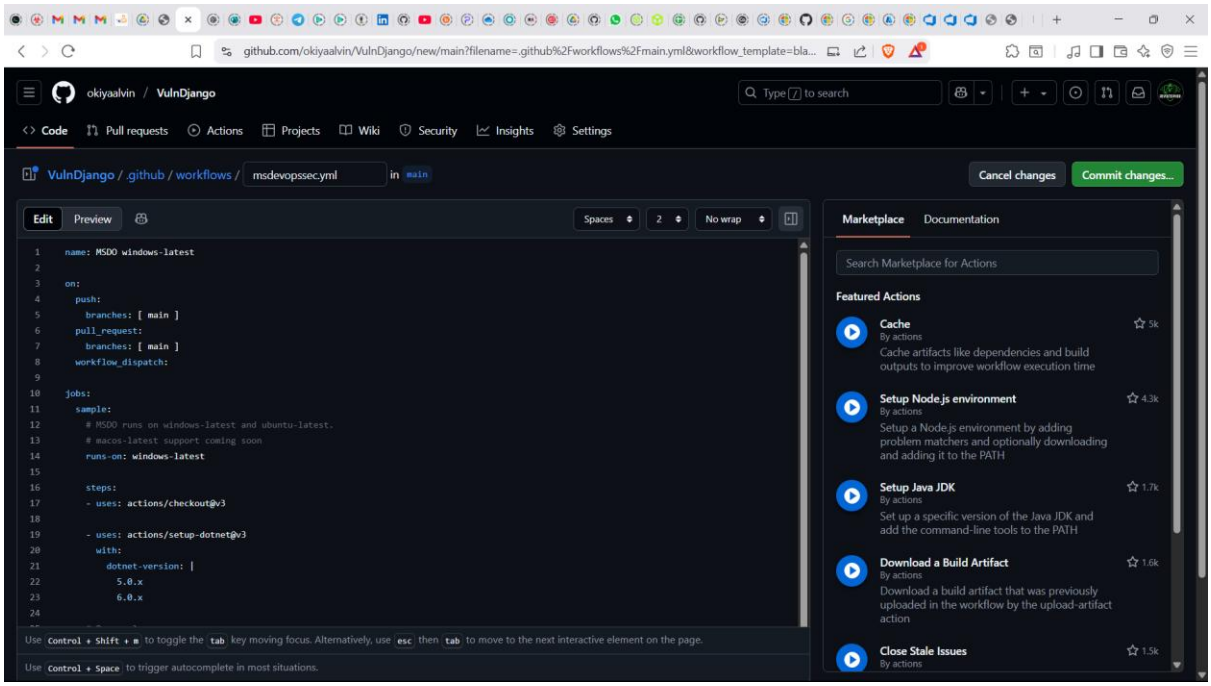




In the textbox I entered a name for my workflow: msdevopssec_me.yml and pasted the following code:

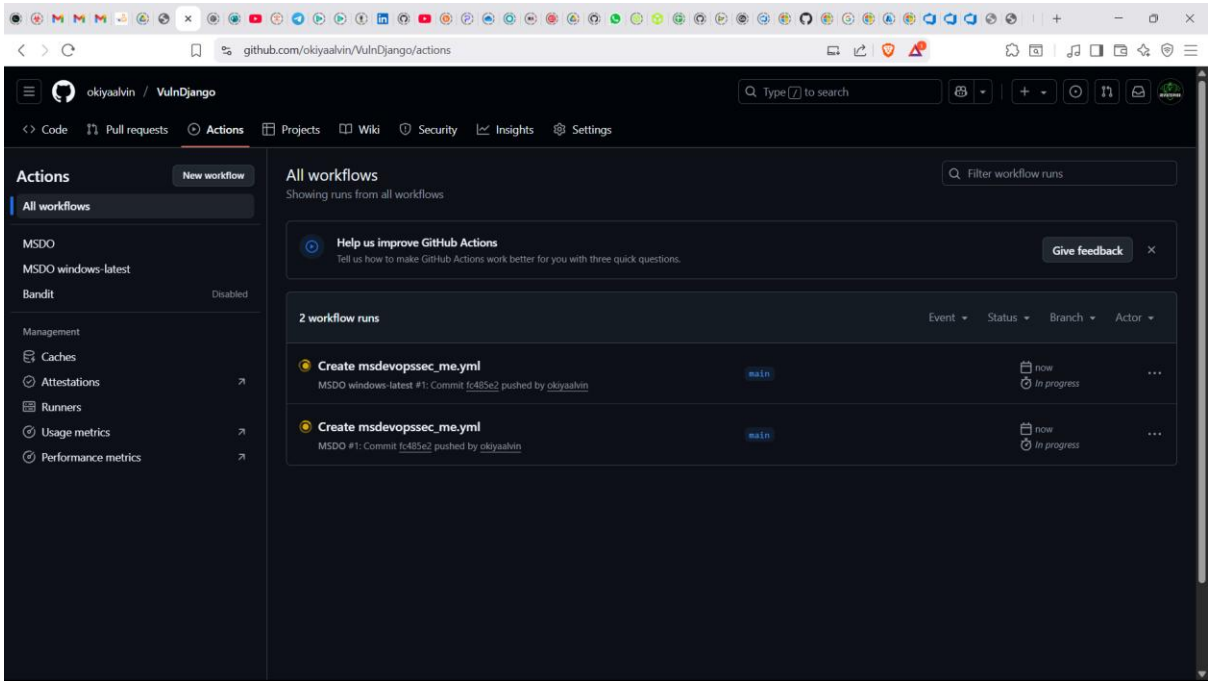
```yaml
name: MSDO windows-latest
    on:
      push:
        branches: [ main ]
      pull_request:
        branches: [ main ]
      workflow_dispatch:

    jobs:
      sample:

        # MSDO runs on windows-latest and ubuntu-latest.
        # macos-latest supporting coming soon
        runs-on: windows-latest

        steps:
        - uses: actions/checkout@v3

        - uses: actions/setup-dotnet@v3
          with:
            dotnet-version: |
              5.0.x
              6.0.x

        # Run analyzers
        - name: Run Microsoft Security DevOps Analysis
          uses: microsoft/security-devops-action@preview
          id: msdo

        # Upload alerts to the Security tab
        - name: Upload alerts to Security tab
          uses: github/codeql-action/upload-sarif@v2
          with:
            sarif_file: ${{ steps.msdo.outputs.sarifFile }}
```

Then clicked on “commit changes”



What the code Does:

- Triggers on pushes, PRs, or manually via workflow_dispatch
- Runs the MSDO CLI using the microsoft/security-devops-action
- Uploads results in SARIF format so GitHub can display alerts in your repo's **Security** tab

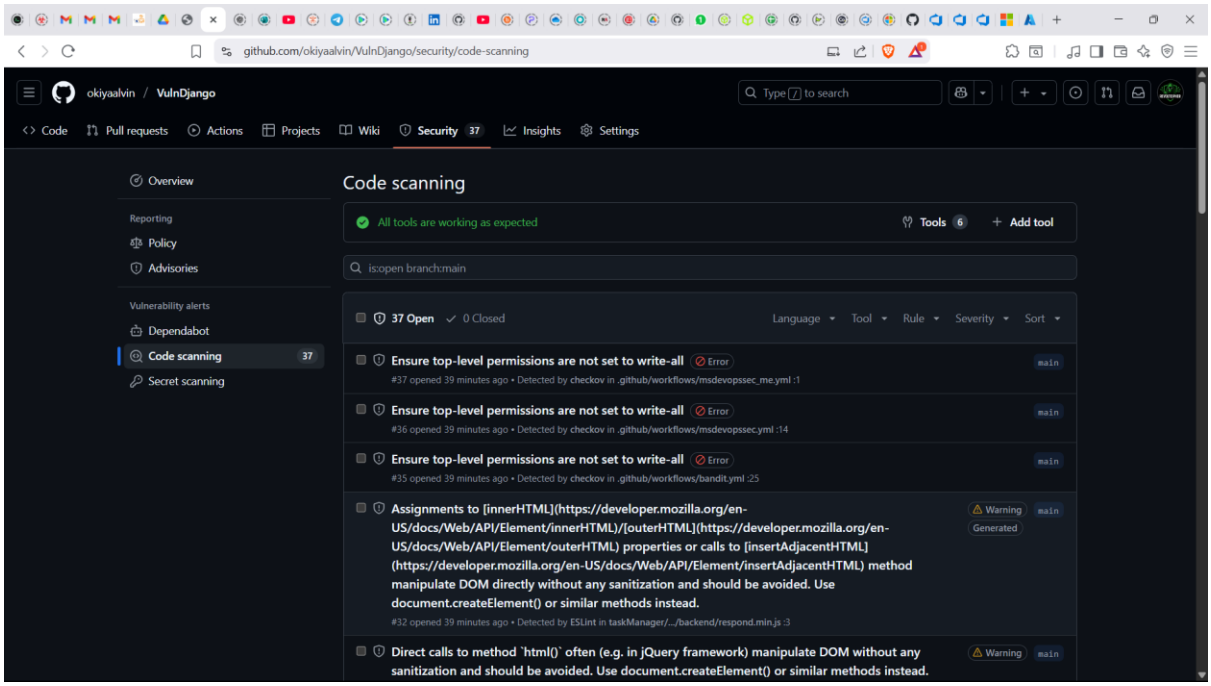


On the actions section, I confirmed that the workflows were running.

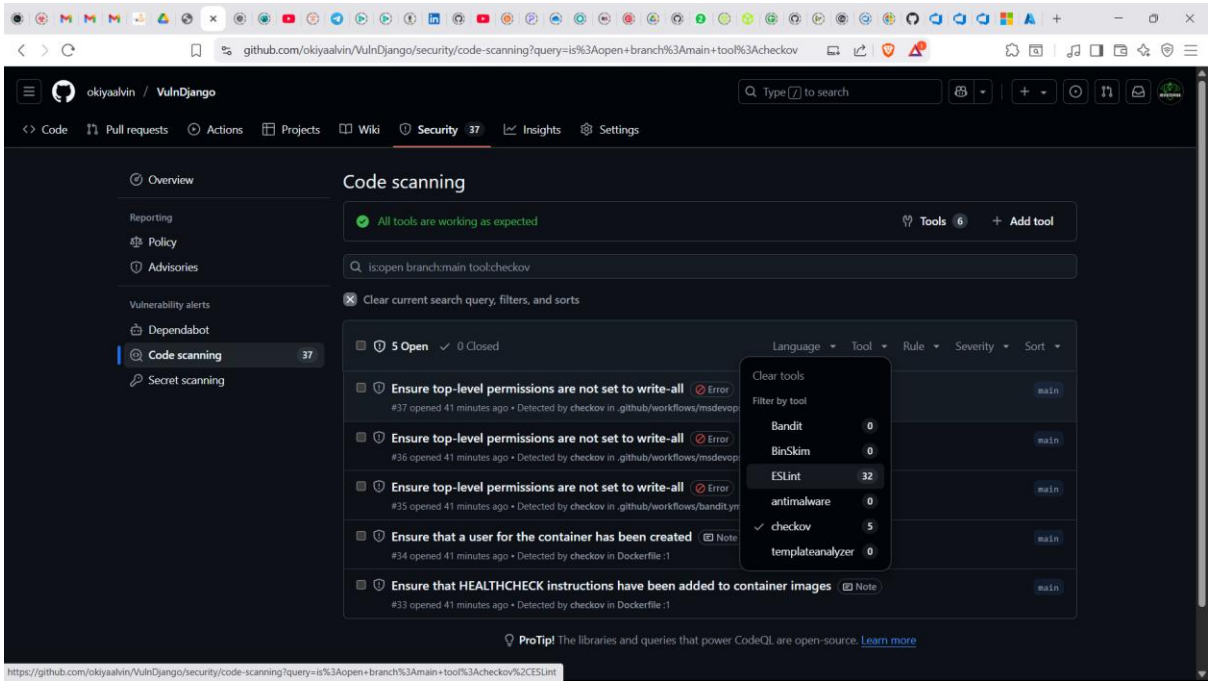
View Scan Results

To view your scan results:

I navigated to **Security** > **Code scanning alerts** > **Tool**.



From the dropdown menu, I selected **Filter by tool**. Filtering by tool allows you to know which tool Identified which vulnerability.



Code scanning findings will be filtered by specific MSDO tools in GitHub. These code scanning results are also pulled into Defender for Cloud recommendations.

Exercise 5: DevOps Security Workbook

Microsoft Defender for Cloud with DevOps integration helps achieve **automated visibility and security scanning across my code, secrets, dependencies, pipelines, and configurations** — without manually inspecting every file, commit, or build.

But lets start knowing what is the **Devops security workbook**

The DevOps Security Workbook Is Your Security Cockpit

It's not just a report — it's an **interactive dashboard** that lets you:

- Investigate credentials exposed in any repo
- View which lines of code triggered vulnerabilities
- See which dependencies are risky or misconfigured
- Prioritize remediation and track improvement over time

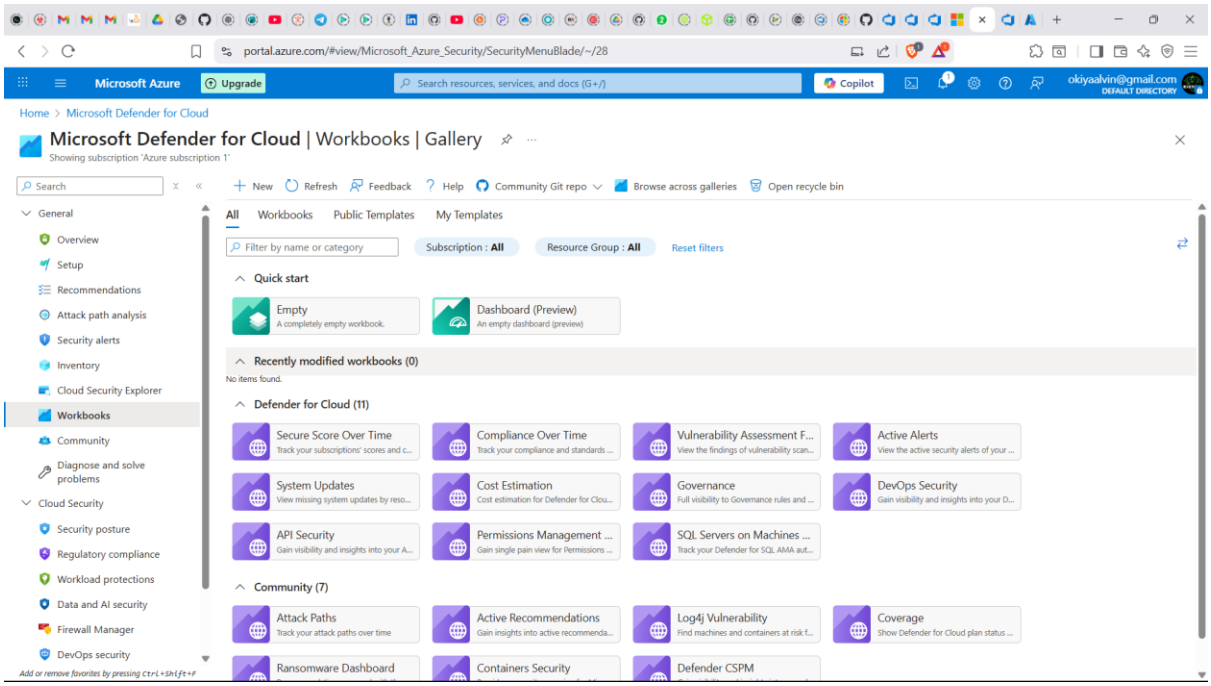
-How It Complements Your MSDO CLI Pipeline

Up to this section, we've already built a robust Kali-based self-hosted CI pipeline with MSDO. Defender for Cloud now expands that by:

- Running similar security logic across all branches and repos — even outside that pipeline
- Giving you long-term visibility across commits, projects, and threats — not just per job

TL;DR: this setup gives you **CI security without limits** — not just in pipelines, but across your whole DevOps ecosystem

To access the workbook, navigate to the Microsoft Defender for Cloud > Workbooks



Conclusion

By completing this assignment, one demonstrates how to operationalize **DevSecOps** practices across multi-repo workflows by unifying local scanning, CI/CD pipeline scanning, and centralized security visibility. Running Bandit locally establishes a developer-first security baseline; enabling the Microsoft Security DevOps extension in Azure DevOps and the MSDO GitHub Action extends that assurance into continuous integration; and reviewing surfaced findings in Defender for Cloud and the DevOps Security Workbook provides an enterprise view for prioritization and response.

The lab reinforces key lessons: shift security left into code, automate repeatable scanning with consistent toolchains, surface results where both developers and security teams can act, and use consolidated reporting to drive remediation at scale. Embedding these practices helps reduce vulnerability exposure windows and strengthens the overall security posture of cloud-hosted applications and pipelines.